Final Laboratory Activity for Lab 3: Complete Role-Based Login System with Registration, Profiles & Admin Management
Course: Web Application Development
Module: REST APIs & Authentication Fundamentals
Duration: 4–5 hours
Total Points: 100 points (+10 bonus)
Format: Individual or Pair

I. Activity Overview
In this capstone laboratory activity, you will build a **complete mock role-based login system** using **Node.js** and **Express** that demonstrates:

- **Full REST API CRUD** (Create, Read, Update, Delete)
- **Public self-registration** for new users
- **Role-based login** with server-controlled redirects (admin / staff / user)
- **User profile management** (edit name, email, password)
- **Admin user management** directly from the dashboard
- **In-memory storage** (no database)

You will test the backend using **Postman** and then consume it from real browser pages using fetch() and sessionStorage. This activity integrates all Module 2 topics: REST APIs, synchronous communication, JSON, authentication, and client–server integration.

II. Learning Objectives
At the end of this activity, you should be able to:

1. **LO1** – Build an Express REST API with full CRUD for a users resource.
2. **LO2** – Add POST /api/login that validates credentials and returns a role-based redirectTo.
3. **LO3** – Add POST /api/register for public self-signup (forced user role).
4. **LO4** – Add PUT /api/profile/:id for self-service profile updates with current-password verification.
5. **LO5** – Build browser pages: login.html, register.html, profile.html, admin.html, staff.html, user.html.
6. **LO6** – Test all endpoints with Postman and the full flow in the browser.
7. **LO7** – Explain the difference between server-side and client-side authentication decisions.

8. **LO8** – Explain why sessionStorage alone is not real authentication.

III. Pre-Lab Questions (10 pts)

Answer the Following Questions: 2pts each

Q1. Why does the server return a redirectTo value instead of letting the client decide where to send the user?

Q2. Since users are stored in memory, what happens when the server restarts? How do you fix this in a real app?

Q3. Why does the login route return the *same* error for unknown email and wrong password?

Q4. Why does sanitizeUser strip the password from every response?

Q5. Is sessionStorage a strong form of protection? Why or why not?

IV. Project Setup

bash

```
mkdir role-login-lab
cd role-login-lab
npm init -y
npm install express
```

Folder structure:

text

```
role-login-lab/
├── server.js
├── package.json
└── public/
    ├── login.html
    ├── register.html
    ├── profile.html
    ├── admin.html
    ├── staff.html
    ├── user.html
    └── style.css
```

V. Source Code

5.1 server.js

```
const express = require('express');
const path = require('path');
const app = express();

// ---------- Middleware ----------
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

// ---------- In-memory "database" ----------
let users = [];
let nextId = 1;

// ---------- Helpers ----------
const findUserById = (id) => users.find(u => u.id === id);

const findUserByEmail = (email) =>
  users.find(u => u.email.toLowerCase() ===
String(email).toLowerCase());
```

```
const sanitizeUser = (user) => {
  const { password, ...safe } = user;
  return safe;
};

const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

// ============================================================
// USER CRUD ROUTES
// ============================================================

// GET /api/users — list all users (no passwords)
app.get('/api/users', (req, res) => {
  res.json(users.map(sanitizeUser));
});

// POST /api/users — admin creates a user with any role
app.post('/api/users', (req, res) => {
  const { name, email, password, role } = req.body;

  if (!name || !email || !password) {
    return res.status(400).json({ error: 'Name, email, and
password are required.' });
  }
  if (!emailRegex.test(email)) {
    return res.status(400).json({ error: 'Invalid email format.'
});
  }
  if (findUserByEmail(email)) {
    return res.status(409).json({ error: 'Email already
registered.' });
  }

  let normalizedRole = 'user';
  if (role === 'admin') normalizedRole = 'admin';
  else if (role === 'staff') normalizedRole = 'staff';

  const newUser = {
    id: nextId++,
    name,
    email,
```

```
    password, // plain text — DEV ONLY
    role: normalizedRole
  };
  users.push(newUser);
  res.status(201).json(sanitizeUser(newUser));
});

// GET /api/users/:id — get one user
app.get('/api/users/:id', (req, res) => {
  const id = parseInt(req.params.id, 10);
  if (isNaN(id)) return res.status(400).json({ error: 'Invalid
user ID.' });

  const user = findUserById(id);
  if (!user) return res.status(404).json({ error: 'User not
found.' });

  res.json(sanitizeUser(user));
});

// PUT /api/users/:id — update a user (admin)
app.put('/api/users/:id', (req, res) => {
  const id = parseInt(req.params.id, 10);
  if (isNaN(id)) return res.status(400).json({ error: 'Invalid
user ID.' });

  const { name, email, password, role } = req.body;

  if (!name || !email) {
    return res.status(400).json({ error: 'Name and email are
required.' });
  }
  if (!emailRegex.test(email)) {
    return res.status(400).json({ error: 'Invalid email format.'
});
  }

  const user = findUserById(id);
  if (!user) return res.status(404).json({ error: 'User not
found.' });
```

```
  const existing = findUserByEmail(email);
  if (existing && existing.id !== id) {
    return res.status(409).json({ error: 'Email already
registered.' });
  }

  user.name = name;
  user.email = email;
  if (password) user.password = password;

  if (role === 'admin') user.role = 'admin';
  else if (role === 'staff') user.role = 'staff';
  else if (role === 'user') user.role = 'user';

  res.json(sanitizeUser(user));
});

// DELETE /api/users/:id — delete a user
app.delete('/api/users/:id', (req, res) => {
  const id = parseInt(req.params.id, 10);
  if (isNaN(id)) return res.status(400).json({ error: 'Invalid
user ID.' });

  const index = users.findIndex(u => u.id === id);
  if (index === -1) return res.status(404).json({ error: 'User
not found.' });

  users.splice(index, 1);
  res.status(204).send();
});

// ============================================================
// PUBLIC REGISTRATION
// ============================================================
app.post('/api/register', (req, res) => {
  const { name, email, password } = req.body;

  if (!name || !email || !password) {
    return res.status(400).json({ error: 'Name, email, and
password are required.' });
  }
```

```
  if (!emailRegex.test(email)) {
    return res.status(400).json({ error: 'Invalid email format.'
});
  }
  if (password.length < 6) {
    return res.status(400).json({ error: 'Password must be at
least 6 characters.' });
  }
  if (findUserByEmail(email)) {
    return res.status(409).json({ error: 'Email already
registered.' });
  }

  const newUser = {
    id: nextId++,
    name,
    email,
    password,
    role: 'user' // forced — clients cannot self-assign
admin/staff
  };

  users.push(newUser);
  res.status(201).json({
    message: 'Registration successful',
    user: sanitizeUser(newUser)
  });
});

// ============================================================
// LOGIN
// ============================================================
app.post('/api/login', (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ error: 'Email and password are
required.' });
  }

  const user = findUserByEmail(email);
```

```
  // Same generic message — don't reveal which failed
  if (!user || user.password !== password) {
    return res.status(401).json({ error: 'Invalid email or
password.' });
  }

  let redirectTo = '/user.html';
  if (user.role === 'admin') redirectTo = '/admin.html';
  else if (user.role === 'staff') redirectTo = '/staff.html';

  res.json({
    message: 'Login successful',
    user: sanitizeUser(user),
    redirectTo
  });
});

// ============================================================
// PROFILE UPDATE (self-service)
// ============================================================
app.put('/api/profile/:id', (req, res) => {
  const id = parseInt(req.params.id, 10);
  if (isNaN(id)) return res.status(400).json({ error: 'Invalid
user ID.' });

  const user = findUserById(id);
  if (!user) return res.status(404).json({ error: 'User not
found.' });

  const { name, email, currentPassword, newPassword } =
req.body;

  if (!name || !email) {
    return res.status(400).json({ error: 'Name and email are
required.' });
  }
  if (!emailRegex.test(email)) {
    return res.status(400).json({ error: 'Invalid email format.'
});
  }
```

```
  const existing = findUserByEmail(email);
  if (existing && existing.id !== id) {
    return res.status(409).json({ error: 'Email already
registered.' });
  }

  // Password change requires current password verification
  if (newPassword) {
    if (!currentPassword) {
      return res.status(400).json({ error: 'Current password is
required to set a new password.' });
    }
    if (user.password !== currentPassword) {
      return res.status(401).json({ error: 'Current password is
incorrect.' });
    }
    if (newPassword.length < 6) {
      return res.status(400).json({ error: 'New password must be
at least 6 characters.' });
    }
    user.password = newPassword;
  }

  user.name = name;
  user.email = email;

  res.json({
    message: 'Profile updated successfully',
    user: sanitizeUser(user)
  });
});

// ============================================================
// START SERVER
// ============================================================
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
  console.log(`Login page:
http://localhost:${PORT}/login.html`);
```

```
  console.log(`Register page:
http://localhost:${PORT}/register.html`);
});
```

5.2 public/login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
  <title>Sign In — Role-Based Lab</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body class="centered">
  <div class="card">
    <h1>Sign In</h1>
    <p class="muted">Use an account created via the register
page or Postman.</p>

    <form id="loginForm" autocomplete="off">
      <label>
        Email
        <input type="email" id="email" placeholder="you@lab.com"
required />
      </label>
      <label>
        Password
        <input type="password" id="password"
placeholder="......" required />
      </label>
      <button type="submit" class="btn primary">Login</button>
    </form>

    <p id="msg" class="msg"></p>

    <p class="muted" style="margin-top:1rem;text-align:center;">
      Don't have an account? <a href="/register.html">Register
here</a>
    </p>
```

```
    <div class="hint">
      <strong>No accounts yet?</strong>
      <div>Create one in Postman first:</div>
      <div><code>POST
http://localhost:3000/api/users</code></div>
      <div>Body: <code>{ "name", "email", "password", "role"
}</code></div>
      <div>Roles: <code>admin</code> · <code>staff</code> ·
<code>user</code></div>
    </div>
  </div>

  <script>
    const form = document.getElementById('loginForm');
    const msg = document.getElementById('msg');

    form.addEventListener('submit', async (e) => {
      e.preventDefault();
      msg.textContent = '';
      msg.className = 'msg';

      const email =
document.getElementById('email').value.trim();
      const password =
document.getElementById('password').value;

      if (!email.includes('@')) {
        msg.textContent = 'Please enter a valid email address.';
        msg.classList.add('error');
        return;
      }
      if (password.length < 6) {
        msg.textContent = 'Password must be at least 6
characters.';
        msg.classList.add('error');
        return;
      }

      try {
        const res = await fetch('/api/login', {
```

```
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ email, password })
        });

        const data = await res.json();

        if (!res.ok) {
          msg.textContent = data.error || 'Login failed.';
          msg.classList.add('error');
          return;
        }

        sessionStorage.setItem('currentUser',
JSON.stringify(data.user));
        window.location.href = data.redirectTo;
      } catch {
        msg.textContent = 'Cannot reach the server. Is it
running?';
        msg.classList.add('error');
      }
    });
  </script>
</body>
</html>
```

5.3 public/register.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
  <title>Register — Role-Based Lab</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body class="centered">
  <div class="card">
    <h1>Create an Account</h1>
```

```
    <p class="muted">Self-registration creates a normal
<strong>user</strong> account.</p>

    <form id="registerForm" autocomplete="off">
      <label>
        Full Name
        <input type="text" id="name" placeholder="Juan Dela
Cruz" required />
      </label>
      <label>
        Email
        <input type="email" id="email" placeholder="you@lab.com"
required />
      </label>
      <label>
        Password
        <input type="password" id="password" placeholder="At
least 6 characters" required />
      </label>
      <label>
        Confirm Password
        <input type="password" id="confirm" placeholder="Re-type
password" required />
      </label>
      <button type="submit" class="btn
primary">Register</button>
    </form>

    <p id="msg" class="msg"></p>

    <p class="muted" style="margin-top:1rem;text-align:center;">
      Already have an account? <a href="/login.html">Back to
Login</a>
    </p>
  </div>

  <script>
    const form = document.getElementById('registerForm');
    const msg = document.getElementById('msg');

    form.addEventListener('submit', async (e) => {
```

```
      e.preventDefault();
      msg.textContent = '';
      msg.className = 'msg';

      const name = document.getElementById('name').value.trim();
      const email =
document.getElementById('email').value.trim();
      const password =
document.getElementById('password').value;
      const confirm = document.getElementById('confirm').value;

      if (!email.includes('@')) {
        msg.textContent = 'Please enter a valid email address.';
        msg.classList.add('error');
        return;
      }
      if (password.length < 6) {
        msg.textContent = 'Password must be at least 6
characters.';
        msg.classList.add('error');
        return;
      }
      if (password !== confirm) {
        msg.textContent = 'Passwords do not match.';
        msg.classList.add('error');
        return;
      }

      try {
        const res = await fetch('/api/register', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ name, email, password })
        });

        const data = await res.json();

        if (!res.ok) {
          msg.textContent = data.error || 'Registration
failed.';
          msg.classList.add('error');
```

```
          return;
        }

        msg.textContent = 'Registration successful! Redirecting
to login...';
        msg.style.color = '#16a34a';

        setTimeout(() => { window.location.href = '/login.html';
}, 1500);
      } catch {
        msg.textContent = 'Cannot reach the server. Is it
running?';
        msg.classList.add('error');
      }
    });
  </script>
</body>
</html>
```

5.4 public/profile.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
  <title>My Profile</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="container">
    <header class="topbar">
      <h1>My Profile</h1>
      <button id="backBtn" class="btn">← Back</button>
    </header>

    <section class="card">
      <h2>Edit Your Information</h2>
      <p class="muted">Leave password fields blank to keep your
current password.</p>

      <form id="profileForm" autocomplete="off">
        <label>
          Full Name
          <input type="text" id="name" required />
        </label>
        <label>
          Email
          <input type="email" id="email" required />
        </label>

        <hr style="margin:1.25rem 0;border:none;border-top:1px
solid #e2e8f0;" />

        <label>
          Current Password <span class="muted">(only if changing
password)</span>
          <input type="password" id="currentPassword"
placeholder="Leave blank to keep current" />
```

```
        </label>
        <label>
          New Password
          <input type="password" id="newPassword"
placeholder="At least 6 characters" />
        </label>

        <button type="submit" class="btn primary">Save
Changes</button>
      </form>

      <p id="msg" class="msg"></p>
    </section>
  </div>

  <script>
    const currentUser =
JSON.parse(sessionStorage.getItem('currentUser'));

    if (!currentUser) {
      window.location.href = '/login.html';
    }

    const form = document.getElementById('profileForm');
    const msg = document.getElementById('msg');

    document.getElementById('name').value = currentUser.name ||
'';
    document.getElementById('email').value = currentUser.email
|| '';

    document.getElementById('backBtn').addEventListener('click',
() => {
      let target = '/user.html';
      if (currentUser.role === 'admin') target = '/admin.html';
      else if (currentUser.role === 'staff') target =
'/staff.html';
      window.location.href = target;
    });

    form.addEventListener('submit', async (e) => {
```

```
      e.preventDefault();
      msg.textContent = '';
      msg.className = 'msg';

      const name = document.getElementById('name').value.trim();
      const email =
document.getElementById('email').value.trim();
      const currentPassword =
document.getElementById('currentPassword').value;
      const newPassword =
document.getElementById('newPassword').value;

      const body = { name, email };
      if (newPassword) {
        body.currentPassword = currentPassword;
        body.newPassword = newPassword;
      }

      try {
        const res = await
fetch(`/api/profile/${currentUser.id}`, {
          method: 'PUT',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify(body)
        });

        const data = await res.json();

        if (!res.ok) {
          msg.textContent = data.error || 'Update failed.';
          msg.classList.add('error');
          return;
        }

        sessionStorage.setItem('currentUser',
JSON.stringify(data.user));

        msg.textContent = 'Profile updated successfully!';
        msg.style.color = '#16a34a';
        document.getElementById('currentPassword').value = '';
        document.getElementById('newPassword').value = '';
```

```
      } catch {
        msg.textContent = 'Cannot reach the server. Is it
running?';
        msg.classList.add('error');
      }
    });
  </script>
</body>
</html>
```

5.5 public/admin.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
  <title>Admin Dashboard</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="container">
    <header class="topbar">
      <h1>Admin Dashboard</h1>
      <div>
        <button id="profileBtn" class="btn">My Profile</button>
        <button id="logoutBtn" class="btn
danger">Logout</button>
      </div>
    </header>

    <section class="card">
      <h2>Welcome, <span id="adminName">...</span></h2>
      <p class="muted">Signed in as <strong
id="adminEmail"></strong> (role: admin)</p>
    </section>

    <section class="card">
      <h2>Registered Users</h2>
      <p class="muted">Manage users directly from this page.</p>
```

```
      <table>
        <thead>
          <tr>

<th>ID</th><th>Name</th><th>Email</th><th>Role</th><th>Actions</
th>
          </tr>
        </thead>
        <tbody id="userTableBody"></tbody>
      </table>
    </section>
  </div>

  <script>
    const currentUser =
JSON.parse(sessionStorage.getItem('currentUser'));

    if (!currentUser || currentUser.role !== 'admin') {
      window.location.href = '/login.html';
    } else {
      document.getElementById('adminName').textContent =
currentUser.name;
      document.getElementById('adminEmail').textContent =
currentUser.email;
    }


document.getElementById('logoutBtn').addEventListener('click',
() => {
      sessionStorage.removeItem('currentUser');
      window.location.href = '/login.html';
    });


document.getElementById('profileBtn').addEventListener('click',
() => {
      window.location.href = '/profile.html';
    });

    let cachedUsers = [];
```

```
    async function loadUsers() {
      try {
        const res = await fetch('/api/users');
        cachedUsers = await res.json();
        const tbody = document.getElementById('userTableBody');

        if (!cachedUsers.length) {
          tbody.innerHTML = '<tr><td colspan="5">No users
yet.</td></tr>';
          return;
        }

        tbody.innerHTML = cachedUsers.map(u => `
          <tr>
            <td>${u.id}</td>
            <td>${u.name}</td>
            <td>${u.email}</td>
            <td><span class="badge
${u.role}">${u.role}</span></td>
            <td>
              <button class="btn small"
onclick="editUser(${u.id})">Edit</button>
              <button class="btn small danger"
onclick="deleteUser(${u.id})">Delete</button>
            </td>
          </tr>
        `).join('');
      } catch {
        document.getElementById('userTableBody').innerHTML =
          '<tr><td colspan="5">Failed to load users.</td></tr>';
      }
    }

    async function deleteUser(id) {
      const user = cachedUsers.find(u => u.id === id);
      if (user && user.id === currentUser.id) {
        alert('You cannot delete your own admin account.');
        return;
      }
      if (!confirm(`Delete user #${id}? This cannot be
undone.`)) return;
```

```
      try {
        const res = await fetch(`/api/users/${id}`, { method:
'DELETE' });
        if (res.status === 204) {
          loadUsers();
        } else {
          const data = await res.json().catch(() => ({}));
          alert(data.error || 'Failed to delete user.');
        }
      } catch {
        alert('Cannot reach the server.');
      }
    }

    async function editUser(id) {
      const user = cachedUsers.find(u => u.id === id);
      if (!user) return;

      const newName = prompt('New name:', user.name);
      if (newName === null) return;

      const newEmail = prompt('New email:', user.email);
      if (newEmail === null) return;

      const newRole = prompt('New role (admin / staff / user):',
user.role);
      if (newRole === null) return;

      try {
        const res = await fetch(`/api/users/${id}`, {
          method: 'PUT',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ name: newName, email: newEmail,
role: newRole })
        });
        const data = await res.json();

        if (res.ok) {
          loadUsers();
        } else {
```

```
          alert(data.error || 'Failed to update user.');
        }
      } catch {
        alert('Cannot reach the server.');
      }
    }

    if (currentUser && currentUser.role === 'admin')
loadUsers();
  </script>
</body>
</html>
```

5.6 public/staff.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
  <title>Staff Dashboard</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="container">
    <header class="topbar">
      <h1>Staff Dashboard</h1>
      <div>
        <button id="profileBtn" class="btn">My Profile</button>
        <button id="logoutBtn" class="btn
danger">Logout</button>
      </div>
    </header>

    <section class="card">
      <h2>Welcome, <span id="staffName">...</span></h2>
      <p class="muted">Signed in as <strong
id="staffEmail"></strong></p>
    </section>
```

```
    <section class="card">
      <h2>Staff Tools</h2>
      <ul>
        <li>Review submitted records</li>
        <li>Update content and announcements</li>
        <li>Reply to user inquiries</li>
      </ul>
      <p class="muted">Staff members cannot manage user
accounts.</p>
    </section>
  </div>

  <script>
    const currentUser =
JSON.parse(sessionStorage.getItem('currentUser'));

    if (!currentUser || currentUser.role !== 'staff') {
      window.location.href = '/login.html';
    } else {
      document.getElementById('staffName').textContent =
currentUser.name;
      document.getElementById('staffEmail').textContent =
currentUser.email;
    }


document.getElementById('logoutBtn').addEventListener('click',
() => {
      sessionStorage.removeItem('currentUser');
      window.location.href = '/login.html';
    });


document.getElementById('profileBtn').addEventListener('click',
() => {
      window.location.href = '/profile.html';
    });
  </script>
</body>
</html>
```

5.7 public/user.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
  <title>User Dashboard</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="container">
    <header class="topbar">
      <h1>User Dashboard</h1>
      <div>
        <button id="profileBtn" class="btn">My Profile</button>
        <button id="logoutBtn" class="btn
danger">Logout</button>
      </div>
    </header>

    <section class="card">
      <h2>Welcome, <span id="userName">...</span></h2>
      <p class="muted">Signed in as <strong
id="userEmail"></strong></p>
    </section>

    <section class="card">
      <h2>Your Account</h2>
      <p>You can update your name, email, and password from the
<strong>My Profile</strong> page.</p>
    </section>
  </div>

  <script>
    const currentUser =
JSON.parse(sessionStorage.getItem('currentUser'));

    if (!currentUser || currentUser.role !== 'user') {
      window.location.href = '/login.html';
```

```
    } else {
      document.getElementById('userName').textContent =
currentUser.name;
      document.getElementById('userEmail').textContent =
currentUser.email;
    }


document.getElementById('logoutBtn').addEventListener('click',
() => {
      sessionStorage.removeItem('currentUser');
      window.location.href = '/login.html';
    });


document.getElementById('profileBtn').addEventListener('click',
() => {
      window.location.href = '/profile.html';
    });
  </script>
</body>
</html>
```

5.8 public/style.css

```
* {
  box-sizing: border-box;
  font-family: system-ui, -apple-system, "Segoe UI", Roboto,
sans-serif;
}

body {
  margin: 0;
  background: #f4f7fb;
  color: #1e293b;
}

.centered {
  min-height: 100vh;
  display: grid;
```

```
  place-items: center;
  padding: 1rem;
}

.container {
  max-width: 960px;
  margin: 0 auto;
  padding: 1.5rem;
}

.card {
  background: #fff;
  border-radius: 14px;
  padding: 1.5rem;
  box-shadow: 0 10px 25px rgba(0, 0, 0, .06);
  margin-bottom: 1.25rem;
}

h1 { margin: 0 0 .25rem; font-size: 1.6rem; }
h2 { margin: 0 0 .5rem; font-size: 1.2rem; }
.muted { color: #64748b; margin: 0 0 1rem; }

label {
  display: block;
  margin-bottom: .85rem;
  font-weight: 600;
  font-size: .9rem;
}

input {
  width: 100%;
  padding: .7rem .85rem;
  border: 1px solid #cbd5e1;
  border-radius: 10px;
  margin-top: .35rem;
  font-size: 1rem;
}

input:focus {
  outline: 2px solid #3b82f6;
  border-color: transparent;
```

```
}

.btn {
  border: 0;
  border-radius: 10px;
  padding: .7rem 1rem;
  font-weight: 700;
  cursor: pointer;
  font-size: .95rem;
  background: #e2e8f0;
  color: #1e293b;
}
.btn:hover { background: #cbd5e1; }

.primary { background: #2563eb; color: #fff; width: 100%; }
.primary:hover { background: #1d4ed8; }

.danger { background: #ef4444; color: #fff; }
.danger:hover { background: #dc2626; }

.btn.small {
  padding: .4rem .7rem;
  font-size: .8rem;
  margin-right: .25rem;
}

.msg { min-height: 1.2rem; font-size: .9rem; margin: .75rem 0 0;
}
.error { color: #dc2626; }

.hint {
  margin-top: 1rem;
  padding: .75rem;
  background: #f8fafc;
  border: 1px dashed #cbd5e1;
  border-radius: 10px;
  font-size: .85rem;
  color: #475569;
}
.hint code {
  background: #e2e8f0;
```

```
  padding: .1rem .3rem;
  border-radius: 5px;
}

.topbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 1.25rem;
}

table { width: 100%; border-collapse: collapse; }
th, td {
  text-align: left;
  padding: .75rem;
  border-bottom: 1px solid #e2e8f0;
}
th {
  background: #f1f5f9;
  font-size: .85rem;
  text-transform: uppercase;
  letter-spacing: .03em;
}

.badge {
  padding: .2rem .5rem;
  border-radius: 999px;
  font-size: .75rem;
  font-weight: 700;
  text-transform: uppercase;
}
.badge.admin { background: #dbeafe; color: #1e40af; }
.badge.staff { background: #fef3c7; color: #92400e; }
.badge.user  { background: #dcfce7; color: #166534; }

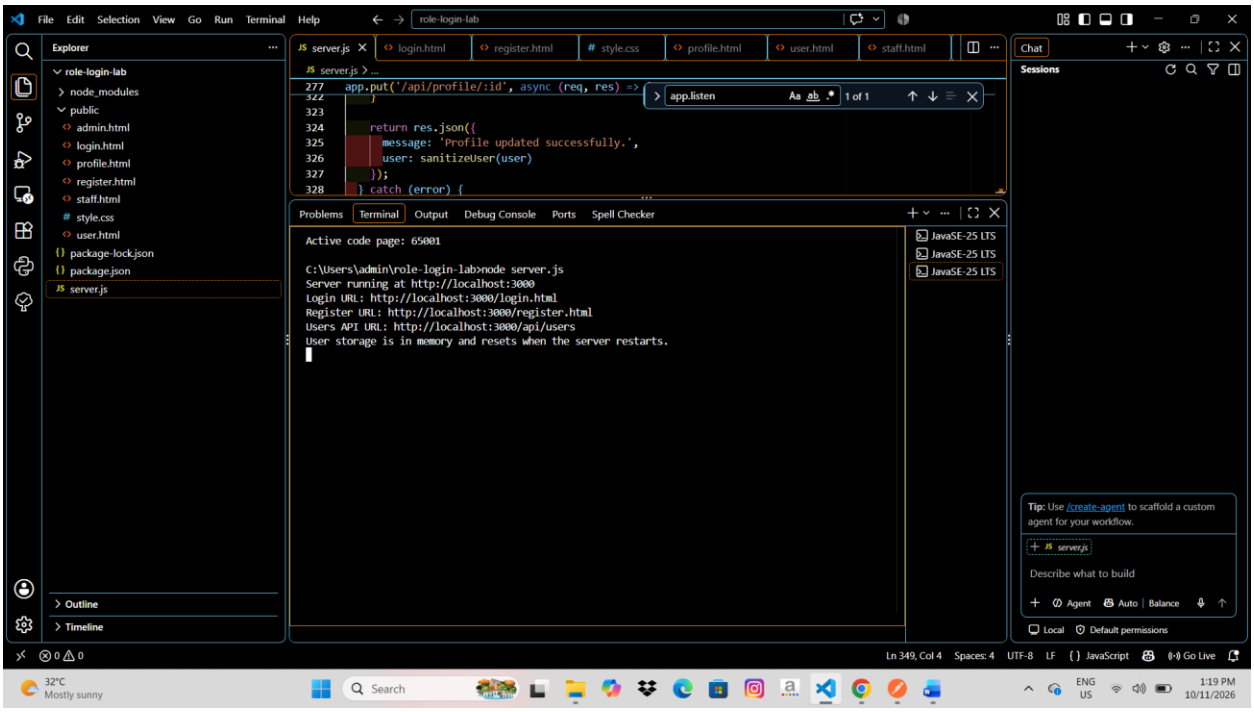
a {
  color: #2563eb;
  text-decoration: none;
  font-weight: 600;
}
a:hover { text-decoration: underline; }
```

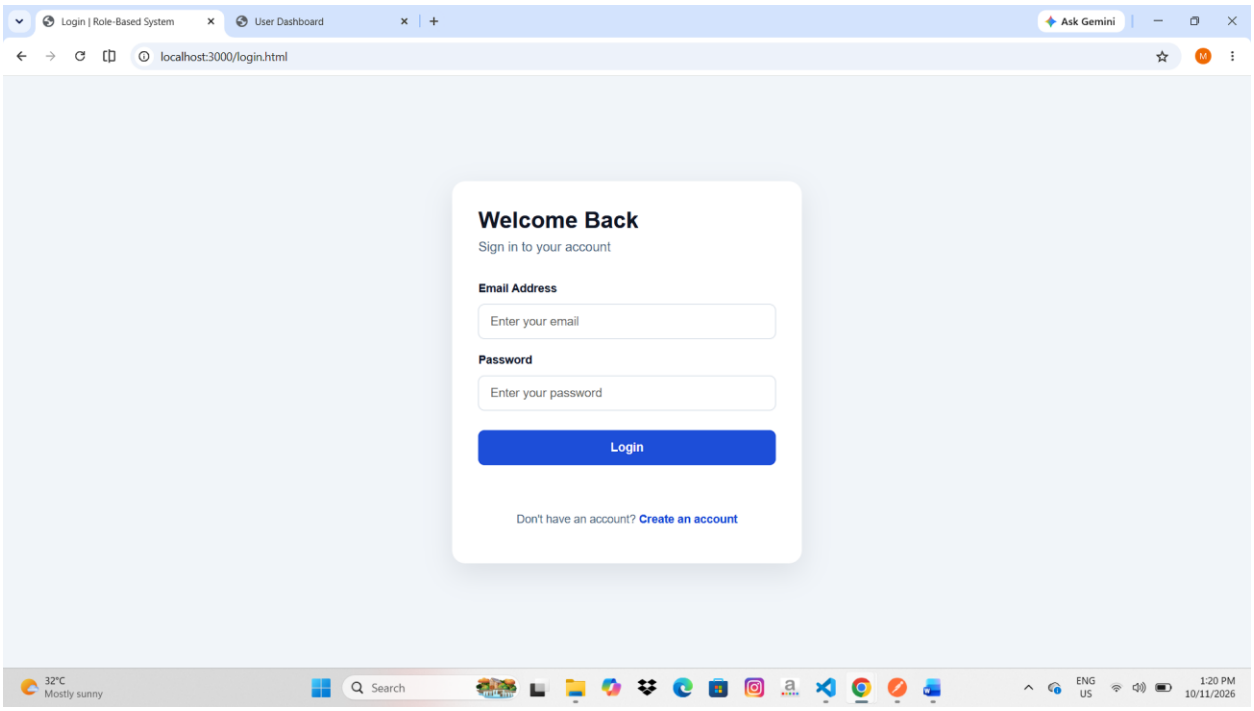
VI. Laboratory Tasks

Task 1 — Run the Server (5 pts)

1. Save all files.
2. Run node server.js.
3. Confirm all three URLs print in the console.
4. Open http://localhost:3000/login.html — the login page should appear.

Deliverable: Screenshot of the terminal and login page.





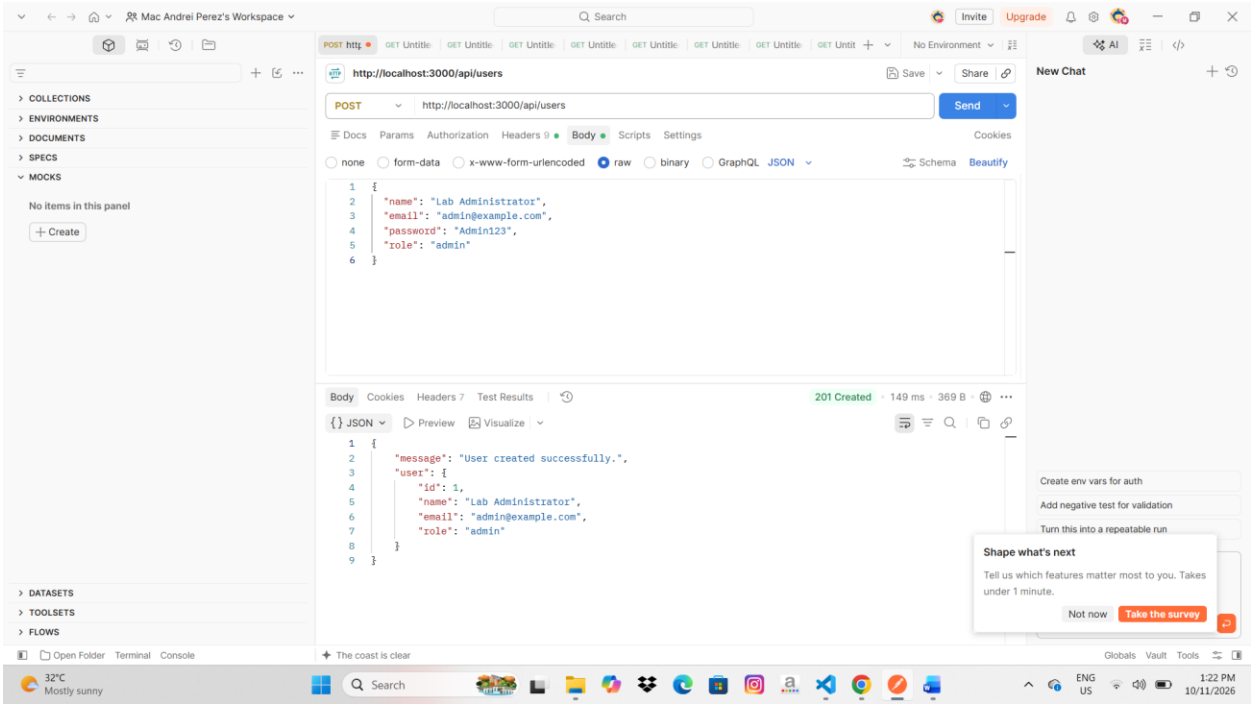
Task 2 — Create Users via Postman (10 pts)

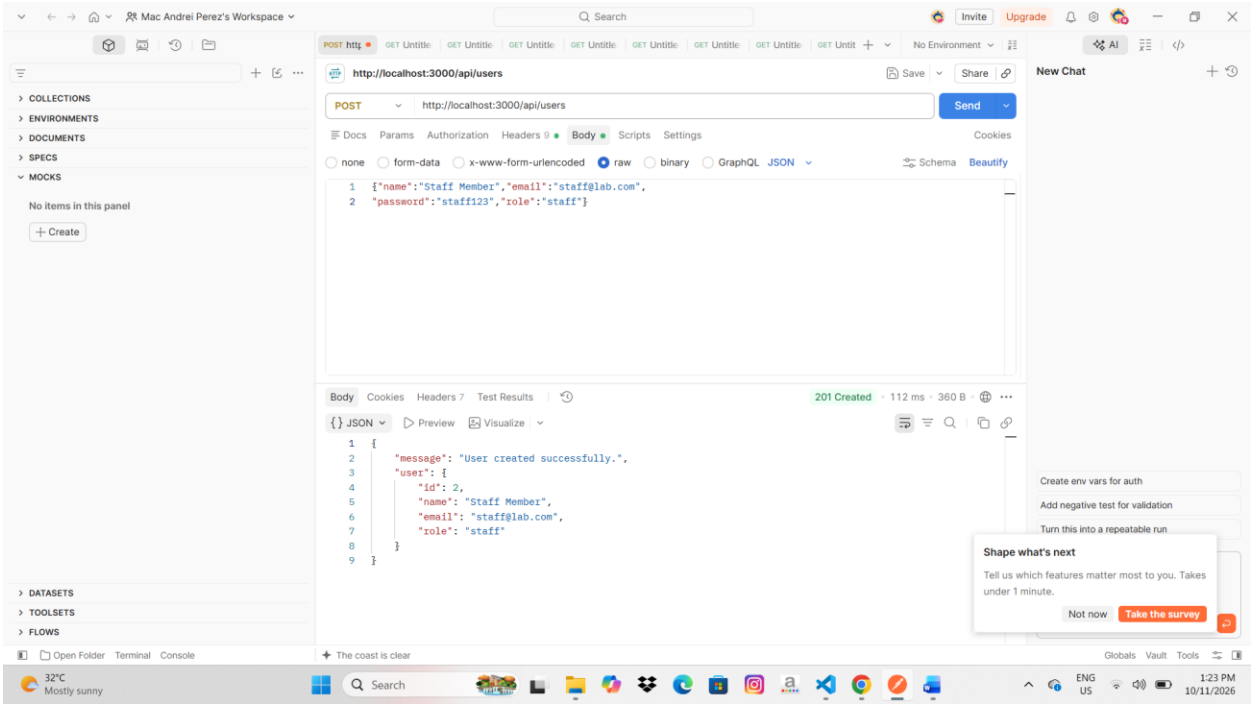
Create **one user of each role** using POST /api/users:

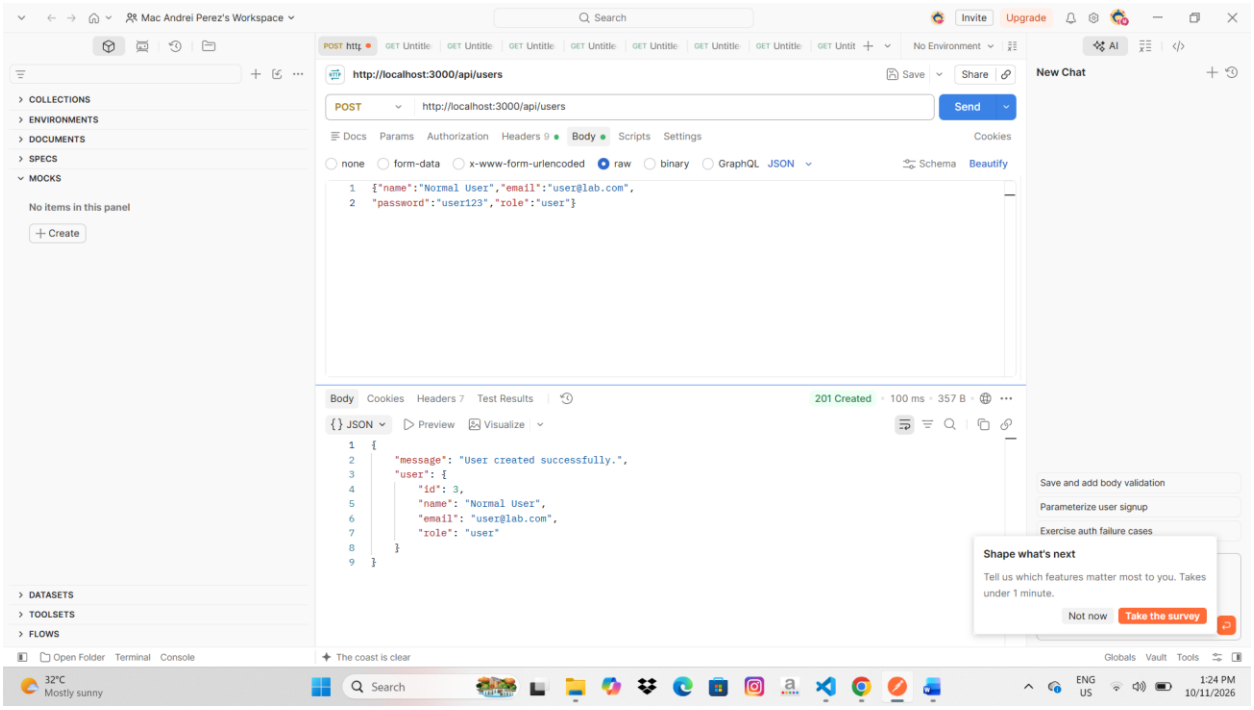
Role	Example Body
admin	{"name":"Admin User","email":"admin@lab.com", "password":"admin123","role":"admin"}
staff	{"name":"Staff Member","email":"staff@lab.com", "password":"staff123","role":"staff"}
user	{"name":"Normal User","email":"user@lab.com", "password":"user123","role":"user"}

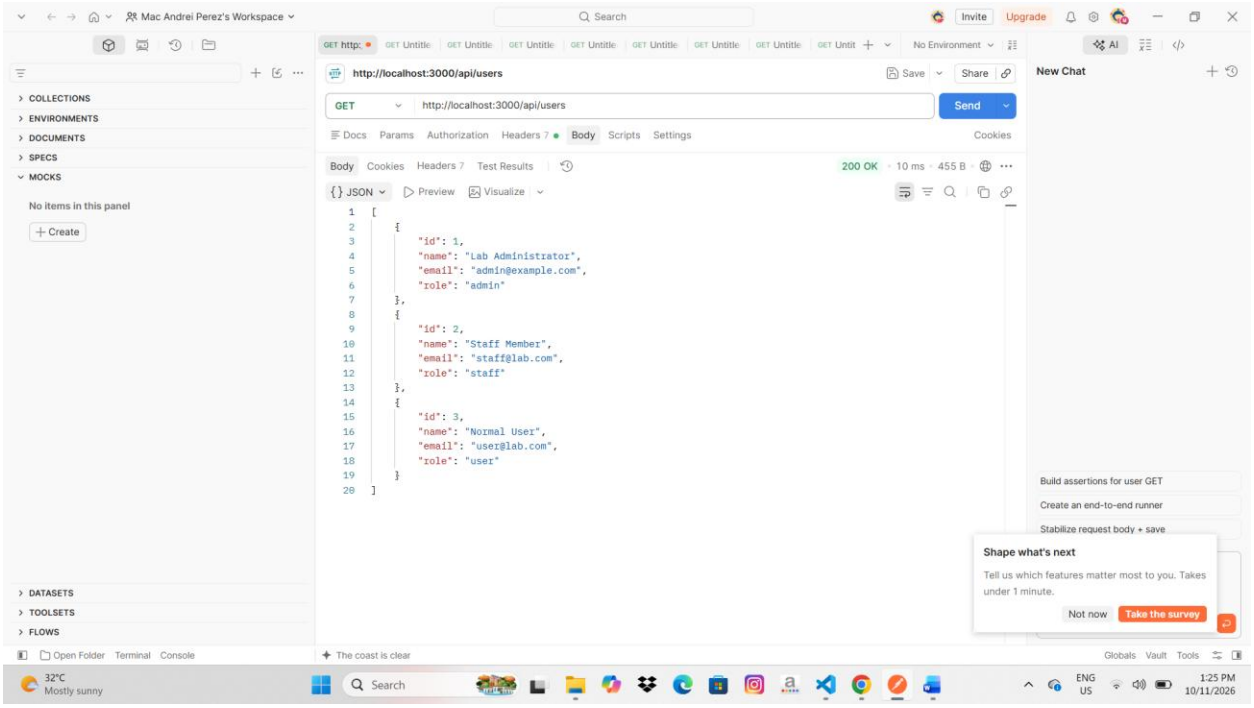
Then run GET /api/users and confirm **no passwords** appear.

Deliverable: Screenshots of all 4 requests.




Mac Andrei Perez's Workspace
Search
Invite
Upgrade
COLLECTIONS
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
No items in this panel
Create
DATASETS
TOOLSETS
FLOWS
http://localhost:3000/api/users
Save
Share
POST
http://localhost:3000/api/users
Send
Docs Params Authorization Headers 9 Body Scripts Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{"name":"Staff Member","email":"staff@lab.com",
"password":"staff123","role":"staff"}
Body Cookies Headers 7 Test Results
201 Created · 112 ms · 360 B
JSON Preview Visualize
{
"message": "User created successfully.",
"user": {
"id": 2,
"name": "Staff Member",
"email": "staff@lab.com",
"role": "staff"
}
}
New Chat
Create env vars for auth
Add negative test for validation
Turn this into a repeatable run
Shape what's next
Tell us which features matter most to you. Takes under 1 minute.
Not now
Take the survey
Open Folder Terminal Console
The coast is clear
Globals Vault Tools
32°C
Mostly sunny
Search
ENG
US
1:23 PM
10/11/2026

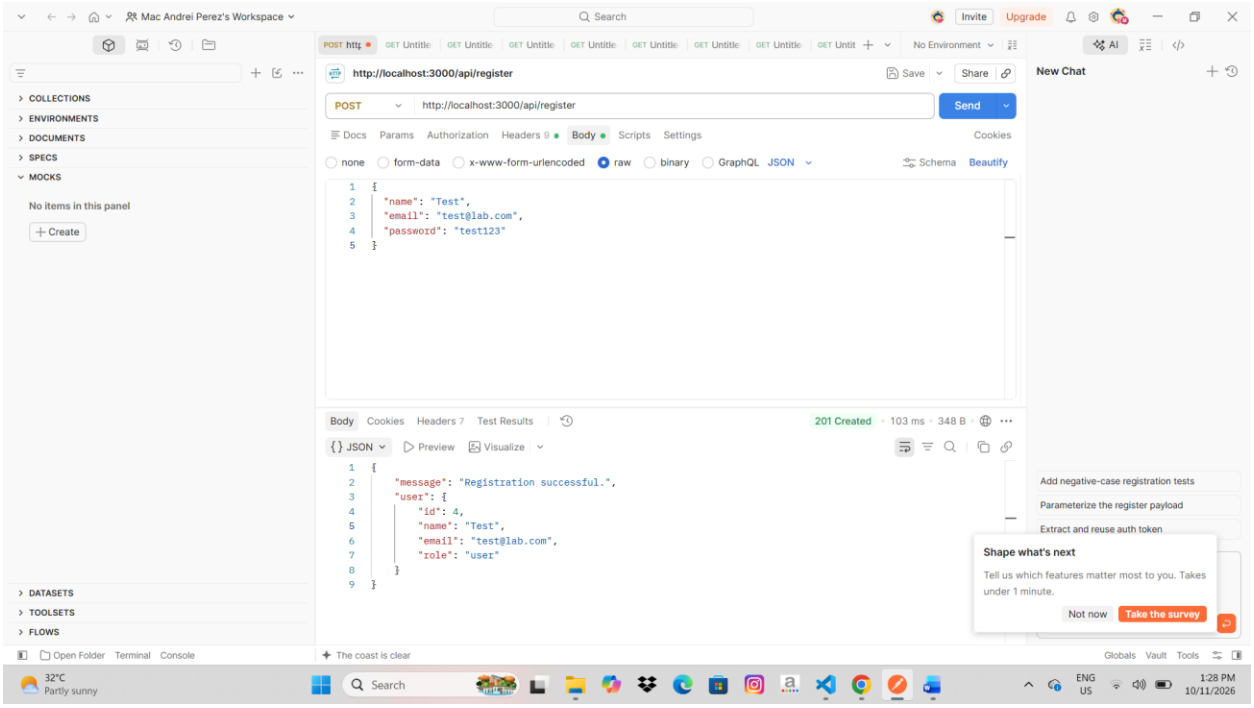

Mac Andrei Perez's Workspace
Search
Invite
Upgrade
COLLECTIONS
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
No items in this panel
Create
DATASETS
TOOLSETS
FLOWS
http://localhost:3000/api/users
Save
Share
POST
http://localhost:3000/api/users
Send
Docs Params Authorization Headers 9 Body Scripts Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{"name":"Normal User","email":"user@lab.com",
"password":"user123","role":"user"}
Body Cookies Headers 7 Test Results
201 Created · 100 ms · 357 B
JSON Preview Visualize
{
"message": "User created successfully.",
"user": {
"id": 3,
"name": "Normal User",
"email": "user@lab.com",
"role": "user"
}
}
New Chat
Save and add body validation
Parameterize user signup
Exercise auth failure cases
Shape what's next
Tell us which features matter most to you. Takes under 1 minute.
Not now
Take the survey
Open Folder Terminal Console
The coast is clear
Globals Vault Tools
32°C
Mostly sunny
Search
ENG
US
1:24 PM
10/11/2026

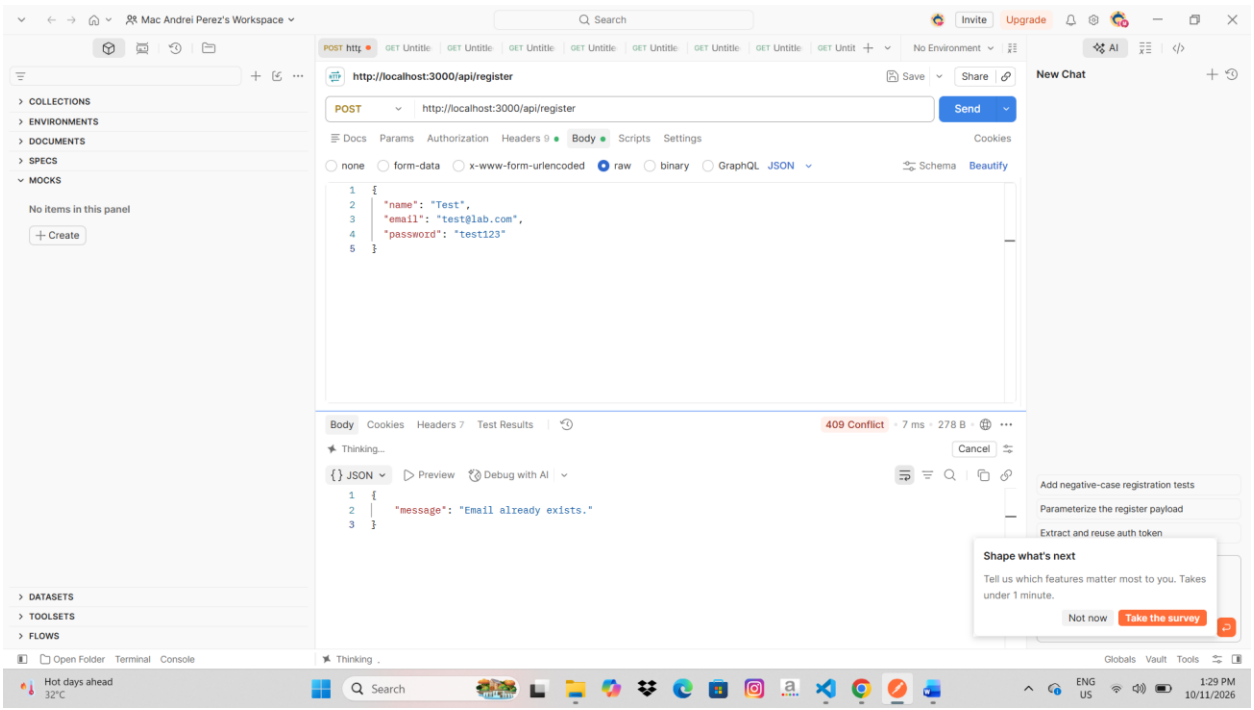


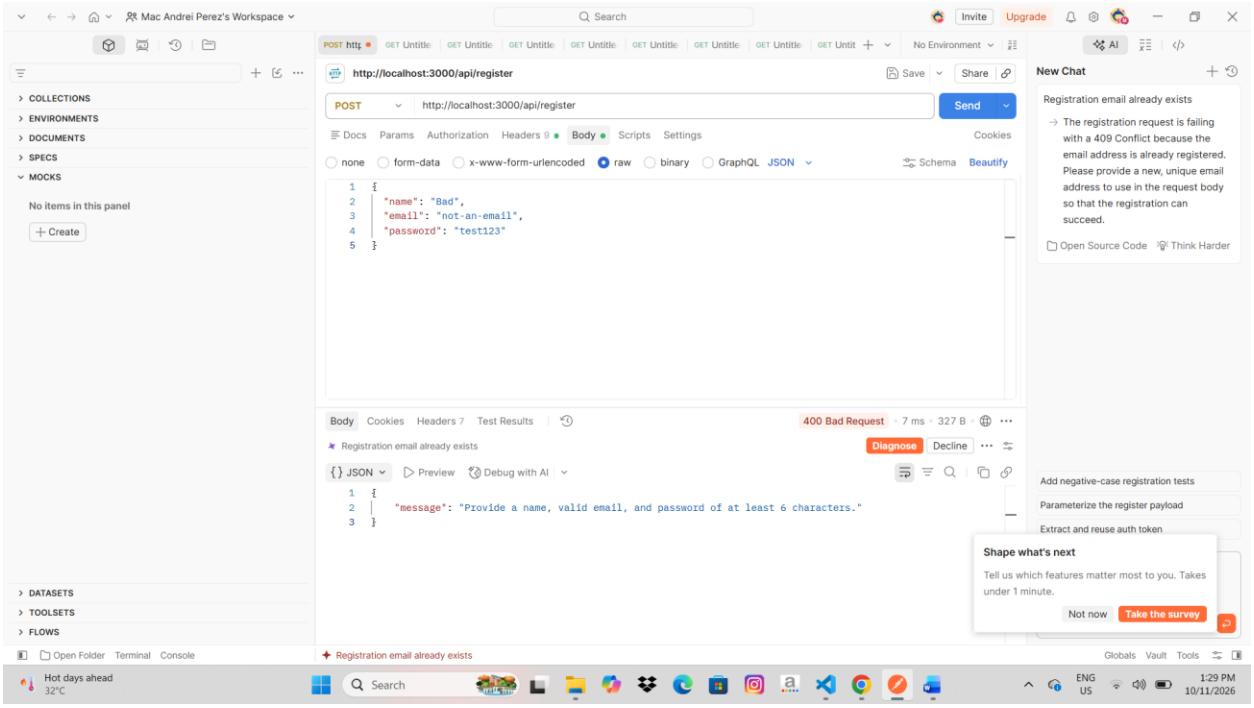
Task 3 — Test Registration in Postman (10 pts)

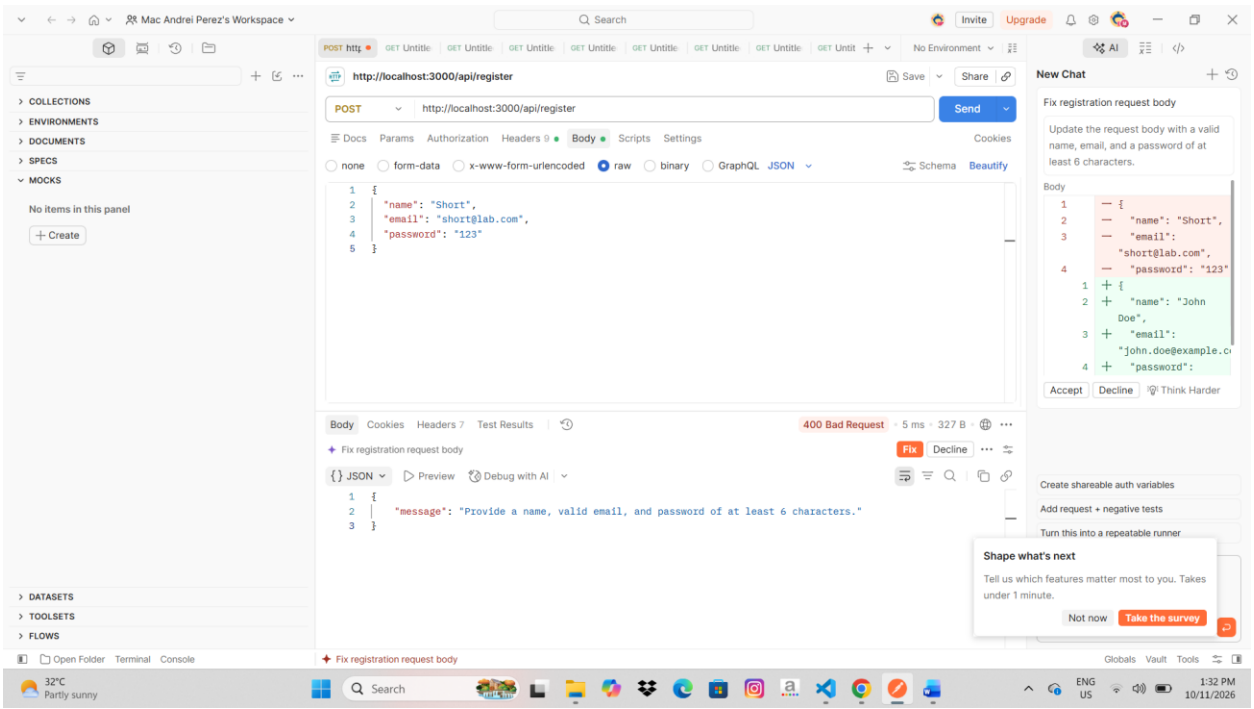
#	Body	Expected
1	{"name":"Test","email":"test@lab.com","password":"test123"}	201
2	Same email again	409
3	{"name":"Bad","email":"not-an-email","password":"test123"}	400
4	{"name":"Short","email":"short@lab.com","password":"123"}	400
5	{"name":"Hacker","email":"hacker@lab.com","password":"hack123","role":"admin"}	201 but role must be **user**

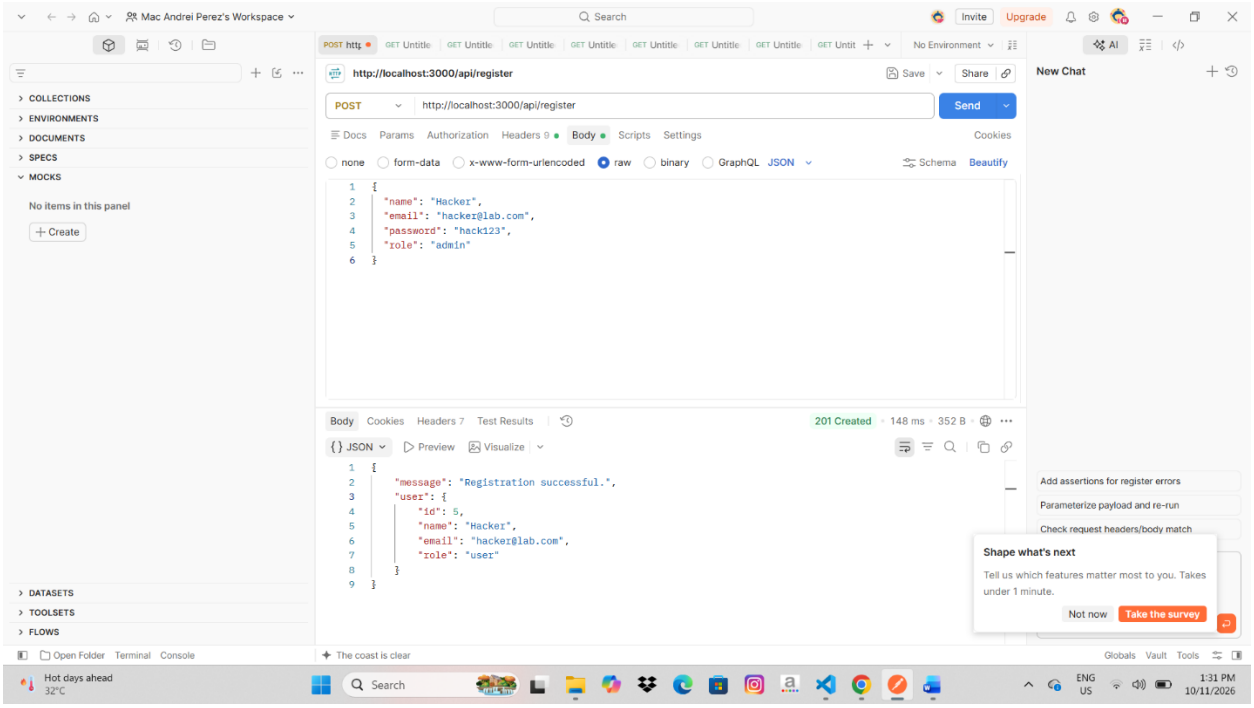
Deliverable: Screenshots of all 5 requests.






Mac Andrei Perez's Workspace
Search
Invite
Upgrade
COLLECTIONS
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
No items in this panel
Create
DATASETS
TOOLSETS
FLOWS
http://localhost:3000/api/register
Save
Share
POST
http://localhost:3000/api/register
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"name": "Bad",
"email": "not-an-email",
"password": "test123"
}
Body Cookies Headers 7 Test Results
400 Bad Request · 7 ms · 327 B
Registration email already exists
Diagnose Decline
JSON Preview Debug with AI
{
"message": "Provide a name, valid email, and password of at least 6 characters."
}
New Chat
Registration email already exists
The registration request is failing with a 409 Conflict because the email address is already registered. Please provide a new, unique email address to use in the request body so that the registration can succeed.
Open Source Code
Think Harder
Add negative-case registration tests
Parameterize the register payload
Extract and reuse auth token
Shape what's next
Tell us which features matter most to you. Takes under 1 minute.
Not now
Take the survey
Open Folder Terminal Console
Registration email already exists
Globals Vault Tools
Hot days ahead
32°C
Search
ENG
US
1:29 PM
10/11/2026

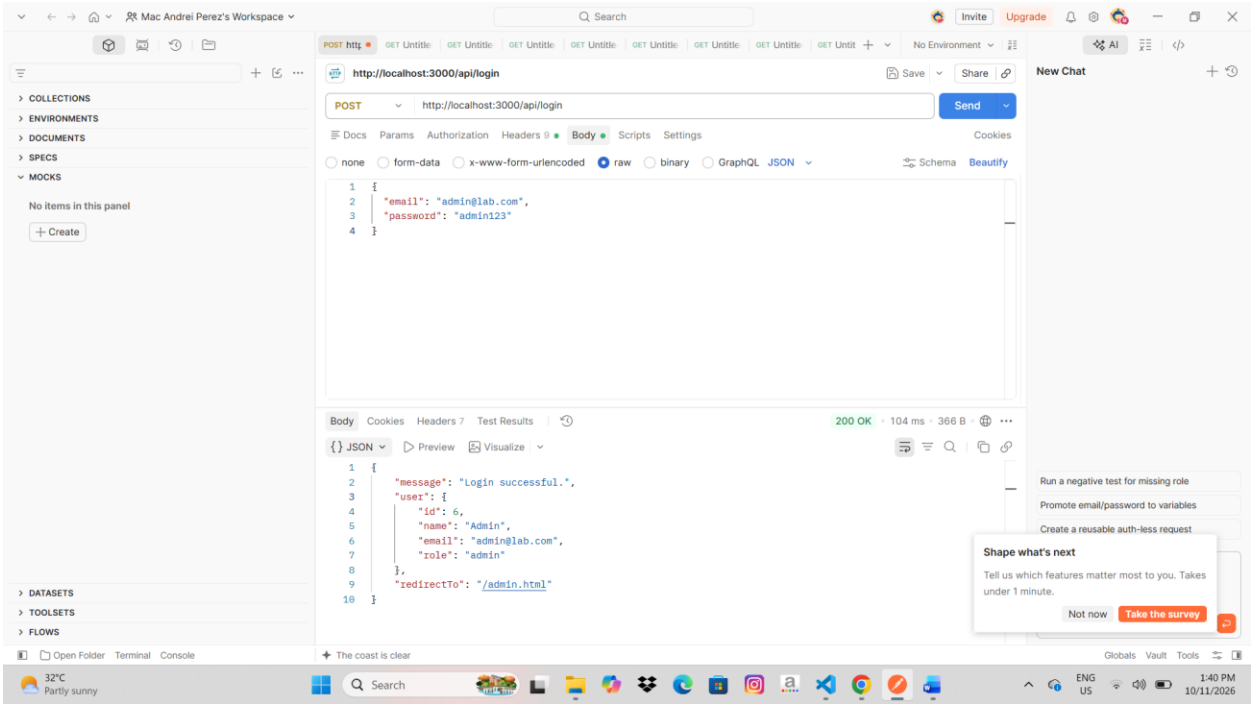

Mac Andrei Perez's Workspace
Search
Invite
Upgrade
COLLECTIONS
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
No items in this panel
Create
DATASETS
TOOLSETS
FLOWS
http://localhost:3000/api/register
Save
Share
POST
http://localhost:3000/api/register
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"name": "Short",
"email": "short@lab.com",
"password": "123"
}
Body Cookies Headers 7 Test Results
400 Bad Request · 5 ms · 327 B
Fix registration request body
Fix Decline
JSON Preview Debug with AI
{
"message": "Provide a name, valid email, and password of at least 6 characters."
}
New Chat
Fix registration request body
Update the request body with a valid name, email, and a password of at least 6 characters.
Body
Accept
Decline
Think Harder
Create shareable auth variables
Add request + negative tests
Turn this into a repeatable runner
Shape what's next
Tell us which features matter most to you. Takes under 1 minute.
Not now
Take the survey
Open Folder Terminal Console
Fix registration request body
Globals Vault Tools
32°C
Partly sunny
Search
ENG
US
1:32 PM
10/11/2026

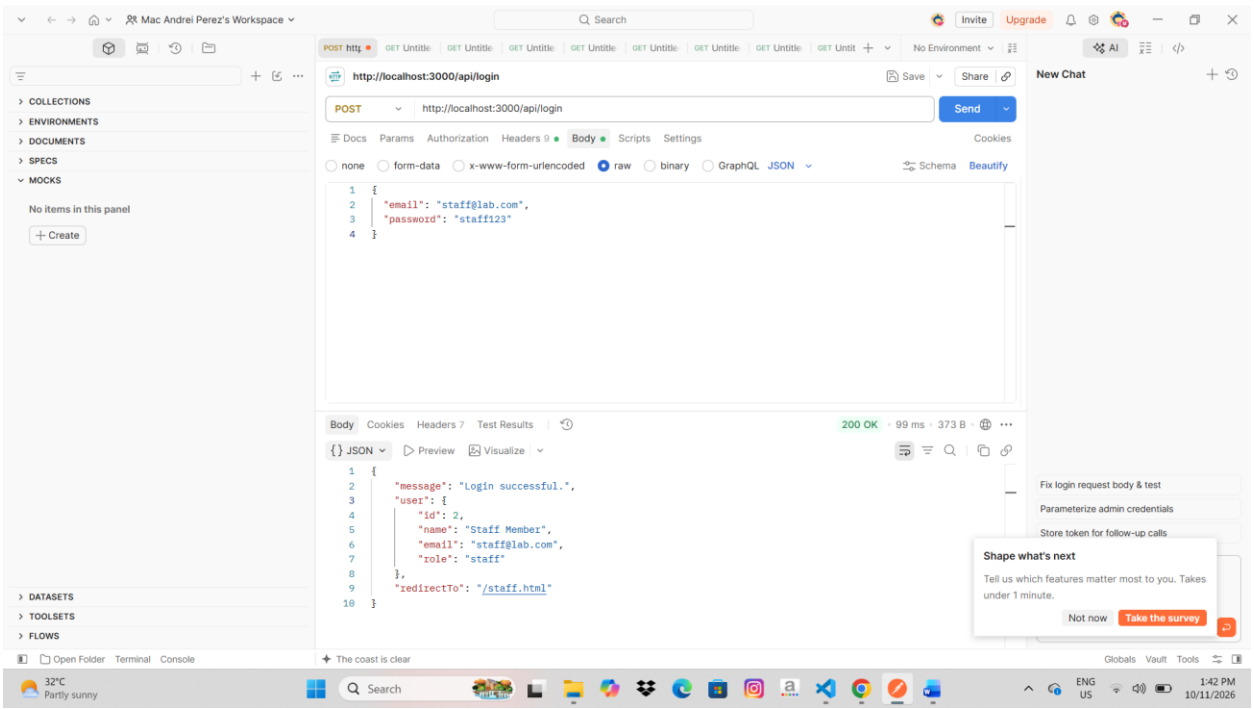


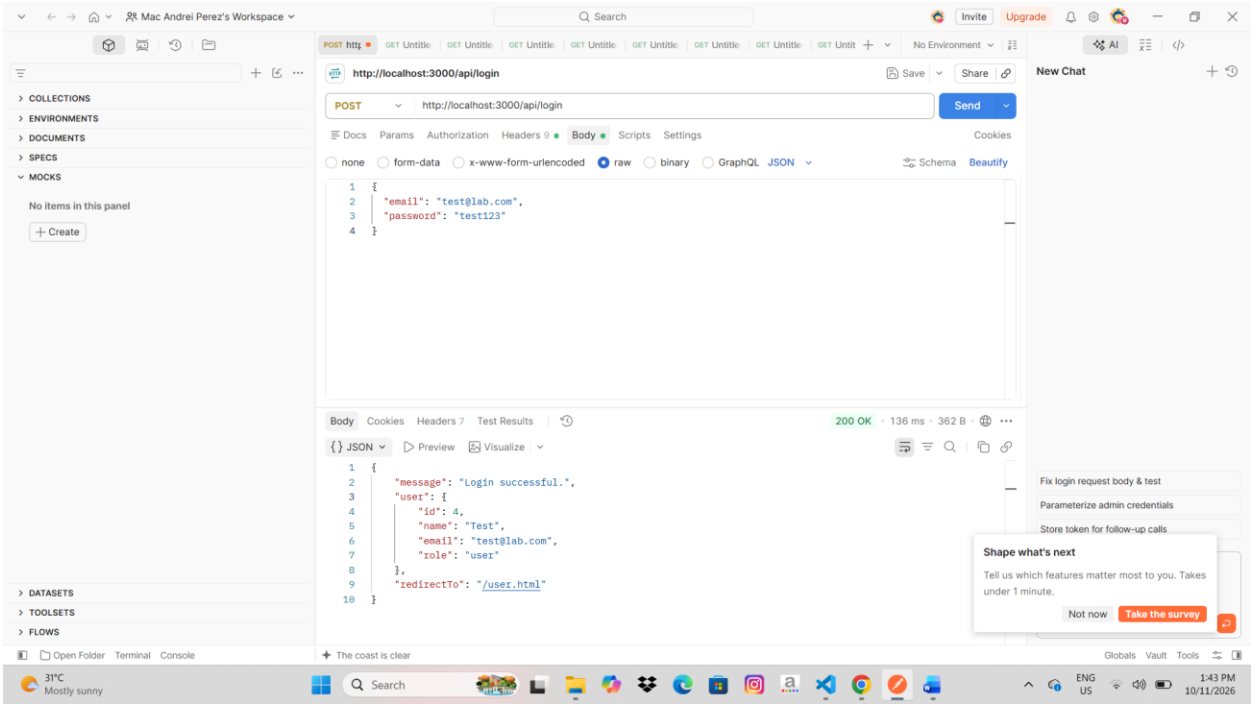
Task 4 — Test Login in Postman (10 pts)

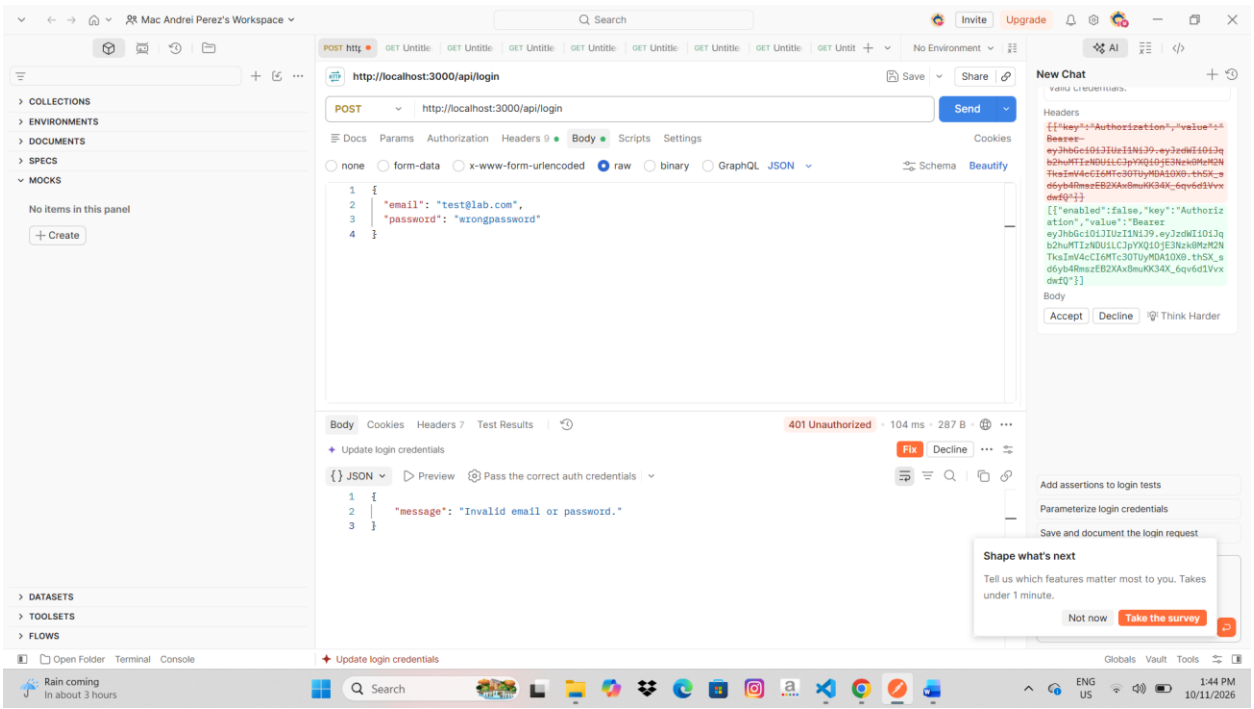
#	Body	Expected
1	admin credentials	200, redirectTo: /admin.html
2	staff credentials	200, redirectTo: /staff.html
3	user credentials	200, redirectTo: /user.html
4	wrong password	401
5	unknown email	401 (same error)
6	missing password	400

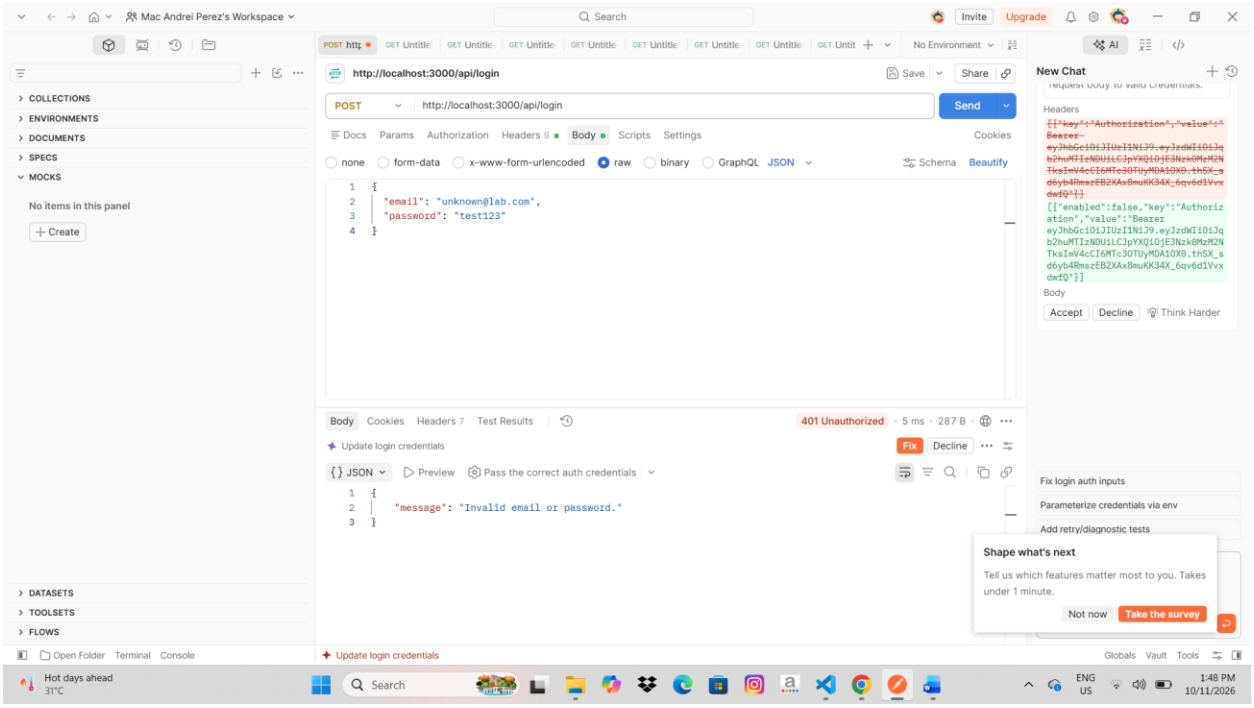
Deliverable: Screenshots of all 6 requests.

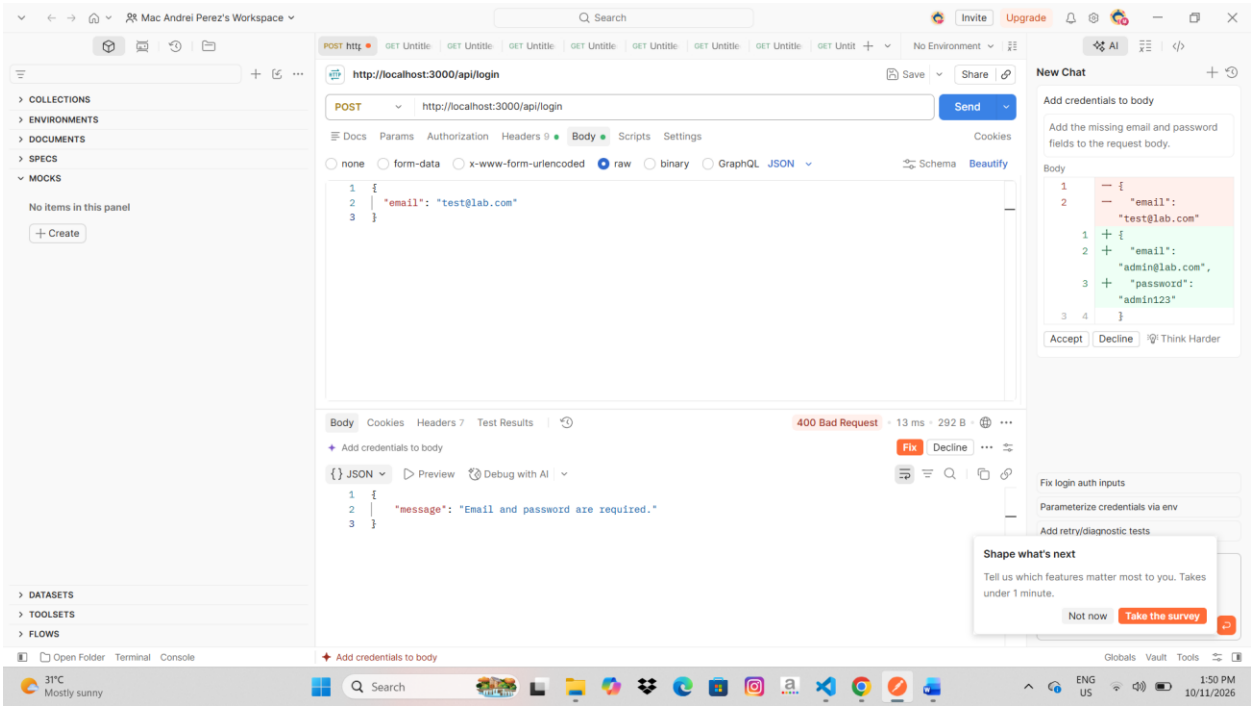

Mac Andrei Perez's Workspace
Search
Invite
Upgrade
http://localhost:3000/api/login
Save
Share
POST
http://localhost:3000/api/login
Send
Docs Params Authorization Headers 9 Body Scripts Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"email": "admin@lab.com",
"password": "admin123"
}
Body Cookies Headers 7 Test Results
200 OK 104 ms 366 B
JSON Preview Visualize
{
"message": "Login successful.",
"user": {
"id": 6,
"name": "Admin",
"email": "admin@lab.com",
"role": "admin"
},
"redirectTo": "/admin.html"
}
COLLECTIONS
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
No items in this panel
Create
DATASETS
TOOLSETS
FLOWS
New Chat
Run a negative test for missing role
Promote email/password to variables
Create a reusable auth-less request
Shape what's next
Tell us which features matter most to you. Takes under 1 minute.
Not now
Take the survey
Open Folder Terminal Console
The coast is clear
Globals Vault Tools
32°C
Partly sunny
Search
ENG
US
1:40 PM
10/11/2026


Mac Andrei Perez's Workspace
Search
Invite
Upgrade
http://localhost:3000/api/login
Save
Share
POST
http://localhost:3000/api/login
Send
Docs Params Authorization Headers 9 Body Scripts Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"email": "staff@lab.com",
"password": "staff123"
}
Body Cookies Headers 7 Test Results
200 OK 99 ms 373 B
JSON Preview Visualize
{
"message": "Login successful.",
"user": {
"id": 2,
"name": "Staff Member",
"email": "staff@lab.com",
"role": "staff"
},
"redirectTo": "/staff.html"
}
COLLECTIONS
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
No items in this panel
Create
DATASETS
TOOLSETS
FLOWS
New Chat
Fix login request body & test
Parameterize admin credentials
Store token for follow-up calls
Shape what's next
Tell us which features matter most to you. Takes under 1 minute.
Not now
Take the survey
Open Folder Terminal Console
The coast is clear
Globals Vault Tools
32°C
Partly sunny
Search
ENG
US
1:42 PM
10/11/2026


Mac Andrei Perez's Workspace
Search
Invite
Upgrade
COLLECTIONS
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
No items in this panel
Create
DATASETS
TOOLSETS
FLOWS
http://localhost:3000/api/login
Save
Share
POST
http://localhost:3000/api/login
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"email": "test@lab.com",
"password": "test123"
}
Body
Cookies
Headers 7
Test Results
200 OK
136 ms
362 B
JSON
Preview
Visualize
{
"message": "Login successful.",
"user": {
"id": 4,
"name": "Test",
"email": "test@lab.com",
"role": "user"
},
"redirectTo": "/user.html"
}
New Chat
Fix login request body & test
Parameterize admin credentials
Store token for follow-up calls
Shape what's next
Tell us which features matter most to you. Takes under 1 minute.
Not now
Take the survey
Open Folder
Terminal
Console
The coast is clear
Globals
Vault
Tools
31°C
Mostly sunny
ENG
US
1:43 PM
10/11/2026


Mac Andrei Perez's Workspace
Search
Invite
Upgrade
COLLECTIONS
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
No items in this panel
Create
DATASETS
TOOLSETS
FLOWS
http://localhost:3000/api/login
Save
Share
POST
http://localhost:3000/api/login
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"email": "test@lab.com",
"password": "wrongpassword"
}
Body
Cookies
Headers 7
Test Results
401 Unauthorized
104 ms
287 B
Update login credentials
Fix
Decline
JSON
Preview
Pass the correct auth credentials
{
"message": "Invalid email or password."
}
New Chat
Headers
Body
Accept
Decline
Think Harder
Add assertions to login tests
Parameterize login credentials
Save and document the login request
Shape what's next
Tell us which features matter most to you. Takes under 1 minute.
Not now
Take the survey
Open Folder
Terminal
Console
Update login credentials
Globals
Vault
Tools
Rain coming
In about 3 hours
ENG
US
1:44 PM
10/11/2026



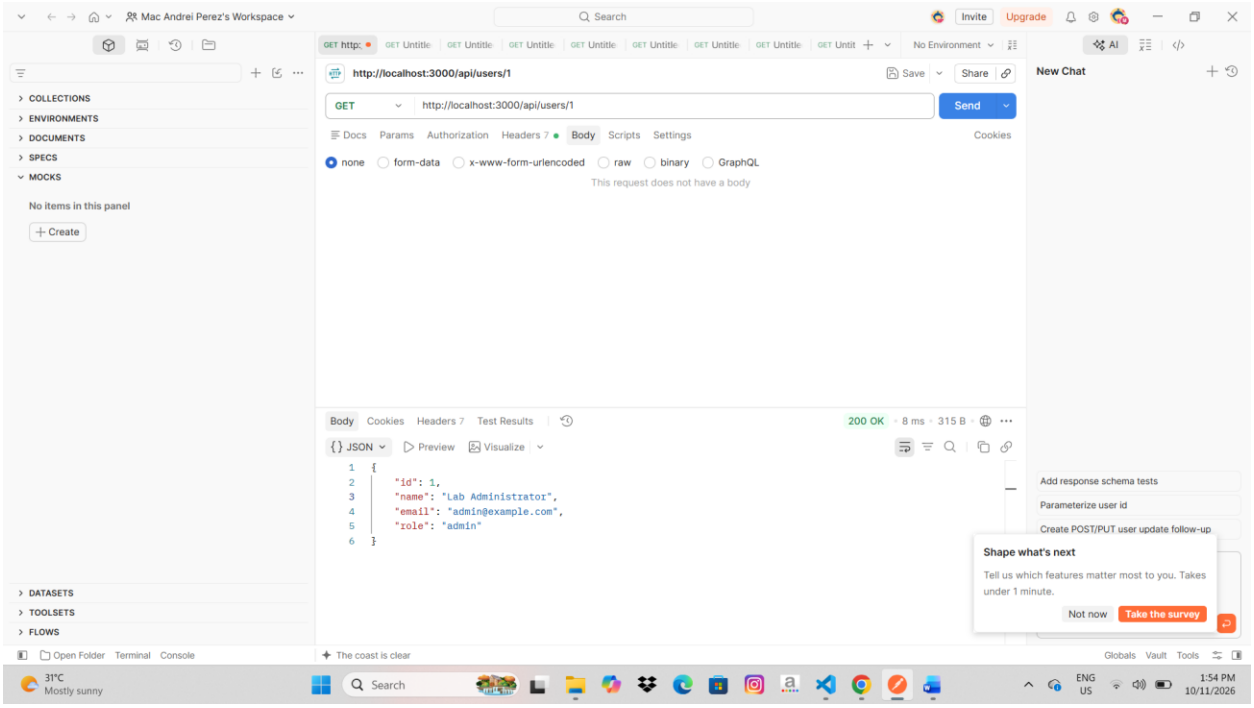


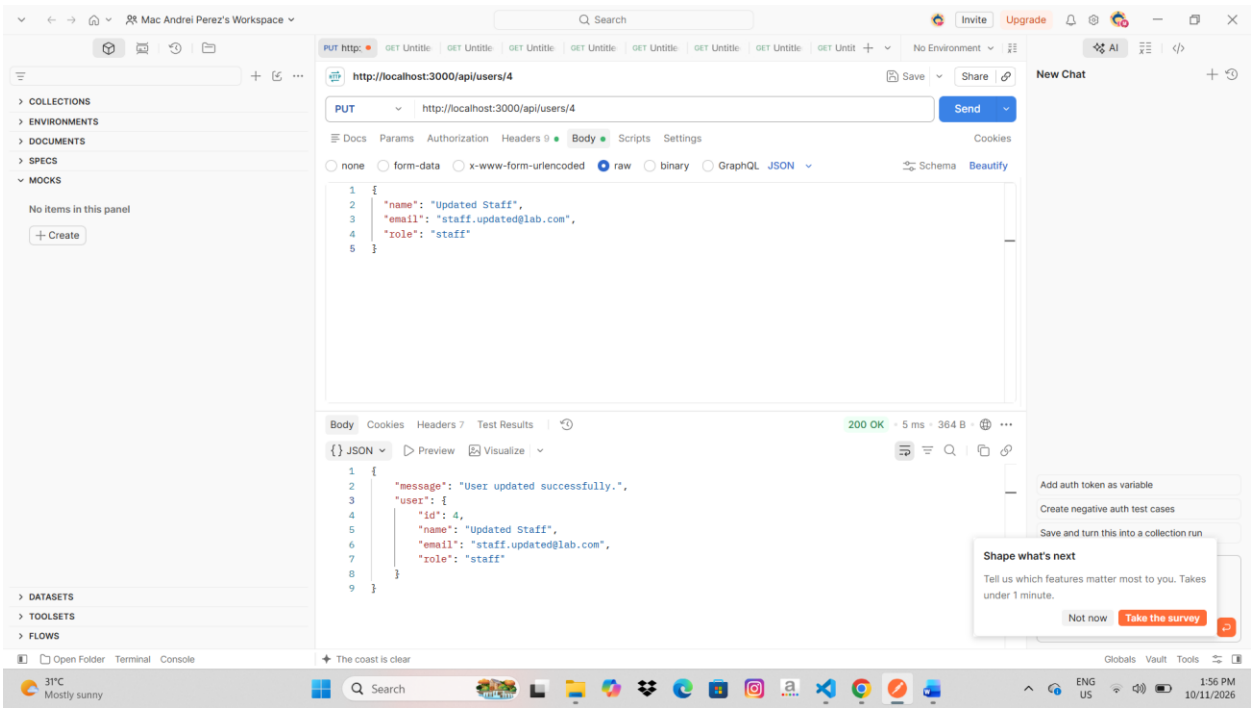
Task 5 — Test Full CRUD in Postman (10 pts)

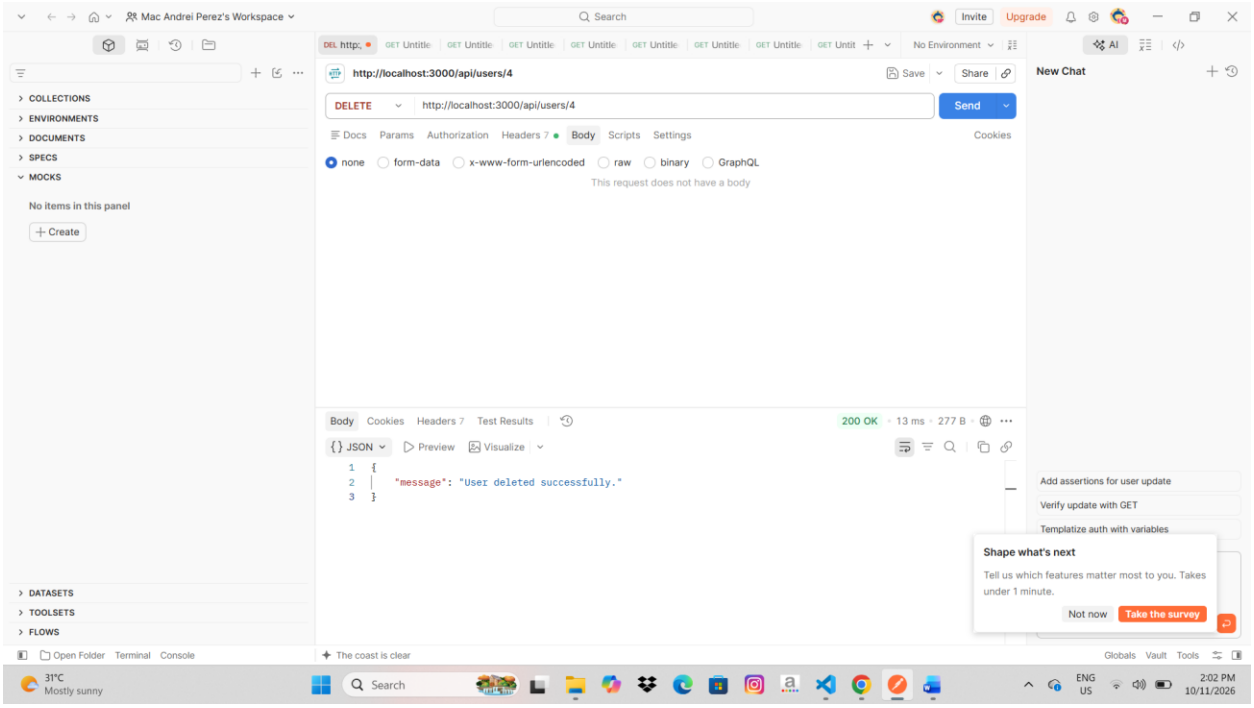
#	Method	URL	Expected
1	GET	/api/users/1	200

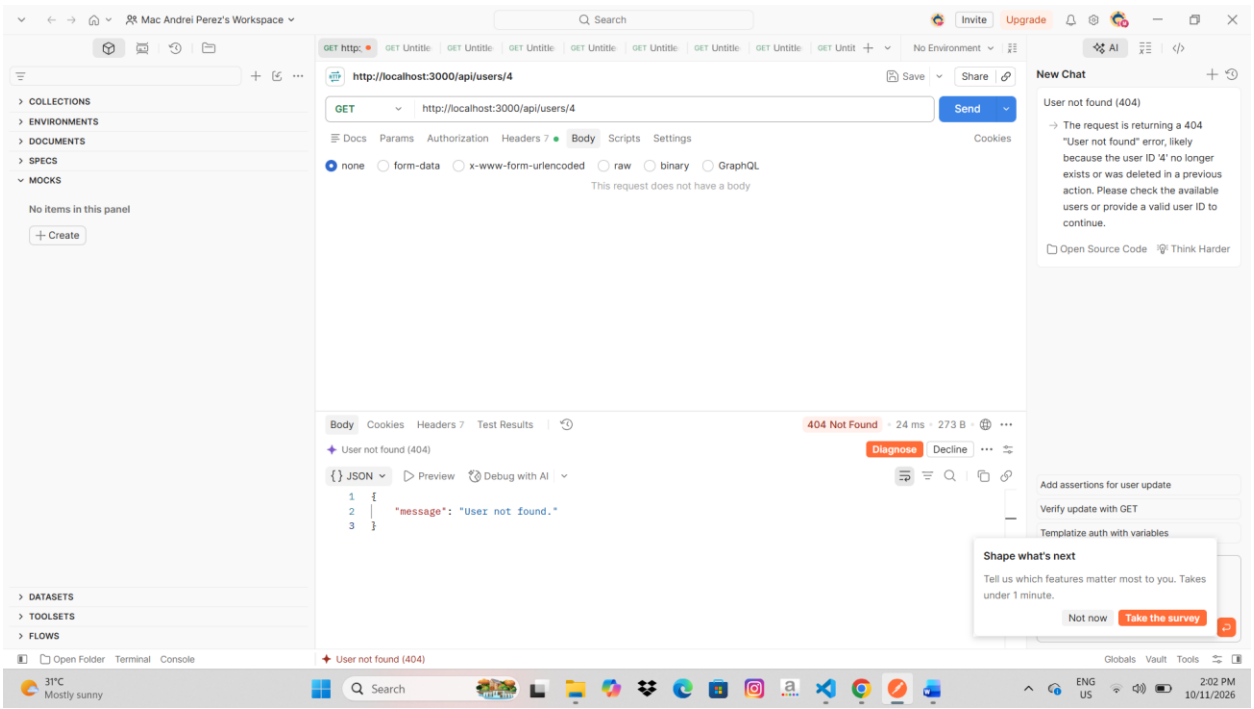
#	Method	URL	Expected
2	PUT	/api/users/4	200 (update name/role)
3	DELETE	/api/users/4	204
4	GET	/api/users/4	404

Deliverable: Screenshots of all 4 requests.


Mac Andrei Perez's Workspace
Search
Invite
Upgrade
http://localhost:3000/api/users/1
Save
Share
GET
http://localhost:3000/api/users/1
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
This request does not have a body
Body
Cookies
Headers 7
Test Results
200 OK
8 ms
315 B
JSON
Preview
Visualize
{
"id": 1,
"name": "Lab Administrator",
"email": "admin@example.com",
"role": "admin"
}
COLLECTIONS
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
No items in this panel
Create
DATASETS
TOOLSETS
FLOWS
New Chat
Add response schema tests
Parameterize user id
Create POST/PUT user update follow-up
Shape what's next
Tell us which features matter most to you. Takes under 1 minute.
Not now
Take the survey
Open Folder
Terminal
Console
The coast is clear
Globals
Vault
Tools
31°C
Mostly sunny
ENG
US
1:54 PM
10/11/2026


Mac Andrei Perez's Workspace
Search
Invite
Upgrade
http://localhost:3000/api/users/4
Save
Share
PUT
http://localhost:3000/api/users/4
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"name": "Updated Staff",
"email": "staff.updated@lab.com",
"role": "staff"
}
Body
Cookies
Headers 7
Test Results
200 OK
5 ms
364 B
JSON
Preview
Visualize
{
"message": "User updated successfully.",
"user": {
"id": 4,
"name": "Updated Staff",
"email": "staff.updated@lab.com",
"role": "staff"
}
}
COLLECTIONS
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
No items in this panel
Create
DATASETS
TOOLSETS
FLOWS
New Chat
Add auth token as variable
Create negative auth test cases
Save and turn this into a collection run
Shape what's next
Tell us which features matter most to you. Takes under 1 minute.
Not now
Take the survey
Open Folder
Terminal
Console
The coast is clear
Globals
Vault
Tools
31°C
Mostly sunny
ENG
US
1:56 PM
10/11/2026

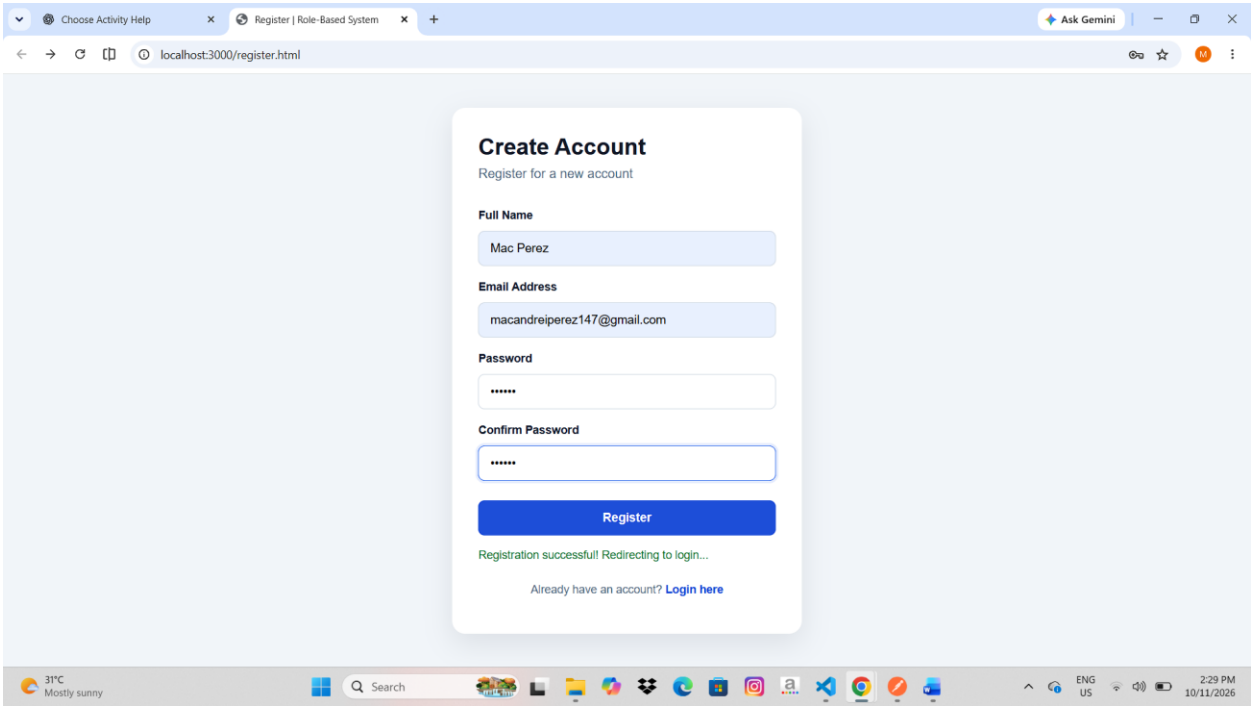

Mac Andrei Perez's Workspace
Search
Invite
Upgrade
No Environment
AI
New Chat
COLLECTIONS
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
No items in this panel
Create
DATASETS
TOOLSETS
FLOWS
http://localhost:3000/api/users/4
Save
Share
DELETE
http://localhost:3000/api/users/4
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
This request does not have a body
Body
Cookies
Headers 7
Test Results
200 OK
13 ms
277 B
JSON
Preview
Visualize
{
"message": "User deleted successfully."
}
Add assertions for user update
Verify update with GET
Templatize auth with variables
Shape what's next
Tell us which features matter most to you. Takes under 1 minute.
Not now
Take the survey
Open Folder
Terminal
Console
The coast is clear
Globals
Vault
Tools
31°C
Mostly sunny
Search
ENG
US
2:02 PM
10/11/2026

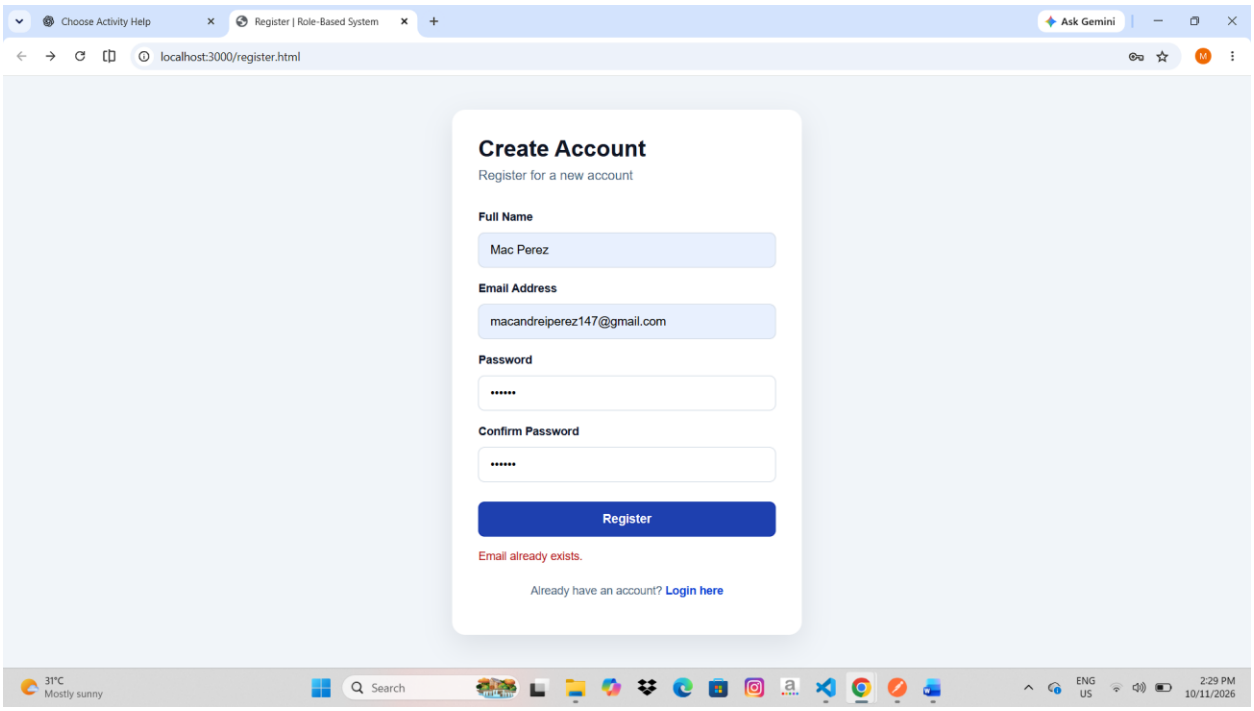

Mac Andrei Perez's Workspace
Search
Invite
Upgrade
No Environment
AI
New Chat
User not found (404)
The request is returning a 404 "User not found" error, likely because the user ID '4' no longer exists or was deleted in a previous action. Please check the available users or provide a valid user ID to continue.
Open Source Code
Think Harder
COLLECTIONS
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
No items in this panel
Create
DATASETS
TOOLSETS
FLOWS
http://localhost:3000/api/users/4
Save
Share
GET
http://localhost:3000/api/users/4
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
This request does not have a body
Body
Cookies
Headers 7
Test Results
404 Not Found
24 ms
273 B
User not found (404)
Diagnose
Decline
JSON
Preview
Debug with AI
{
"message": "User not found."
}
Add assertions for user update
Verify update with GET
Templatize auth with variables
Shape what's next
Tell us which features matter most to you. Takes under 1 minute.
Not now
Take the survey
Open Folder
Terminal
Console
User not found (404)
Globals
Vault
Tools
31°C
Mostly sunny
Search
ENG
US
2:02 PM
10/11/2026

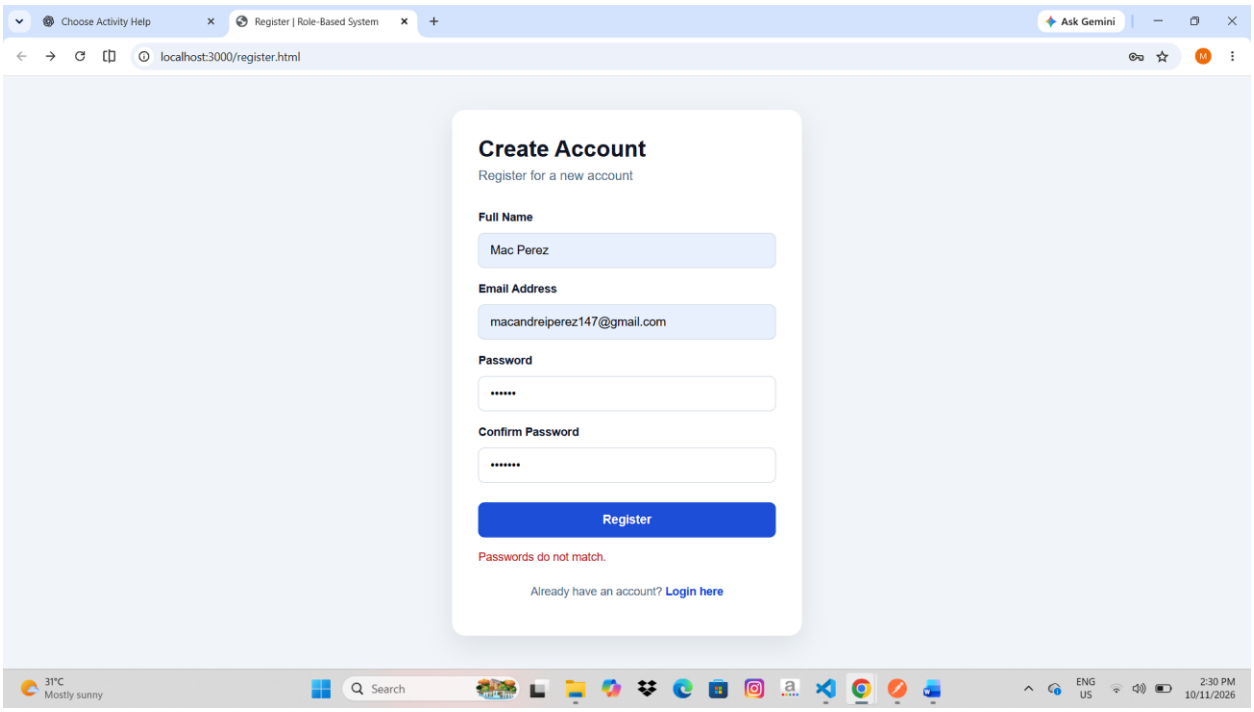
Task 6 — Browser Registration Flow (10 pts)

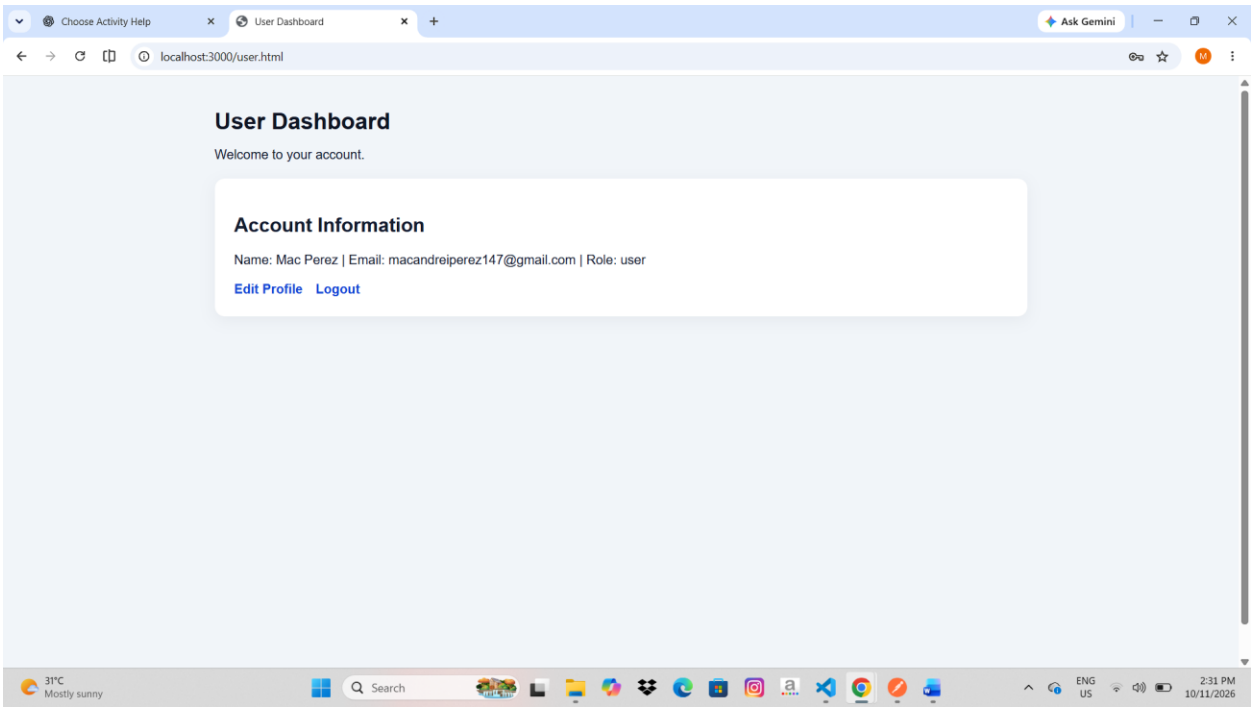
1. Open http://localhost:3000/register.html.
2. Register a new account → success message → redirect to login.
3. Try registering the same email → duplicate error.
4. Try mismatched passwords → error.
5. Log in with the new account → should land on /user.html.

Deliverable: Screenshots of successful registration, duplicate error, and user dashboard.




Choose Activity Help
Register | Role-Based System
Ask Gemini
localhost:3000/register.html
Create Account
Register for a new account
Full Name
Mac Perez
Email Address
macandreiperez147@gmail.com
Password
Confirm Password
Register
Email already exists.
Already have an account? Login here
31°C
Mostly sunny
Search
ENG
US
2:29 PM
10/11/2026


Choose Activity Help
Register | Role-Based System
Ask Gemini
localhost:3000/register.html
Create Account
Register for a new account
Full Name
Mac Perez
Email Address
macandreiperez147@gmail.com
Password
Confirm Password
Register
Passwords do not match.
Already have an account? Login here
31°C
Mostly sunny
Search
ENG
US
2:30 PM
10/11/2026

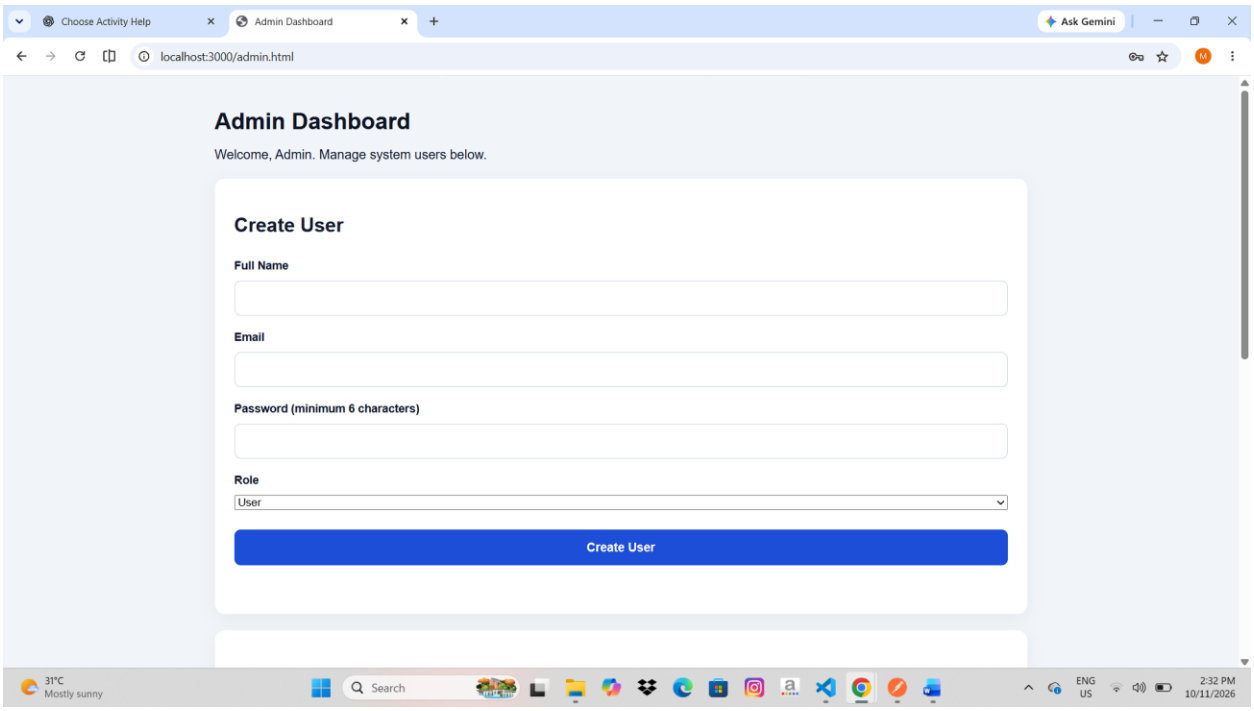


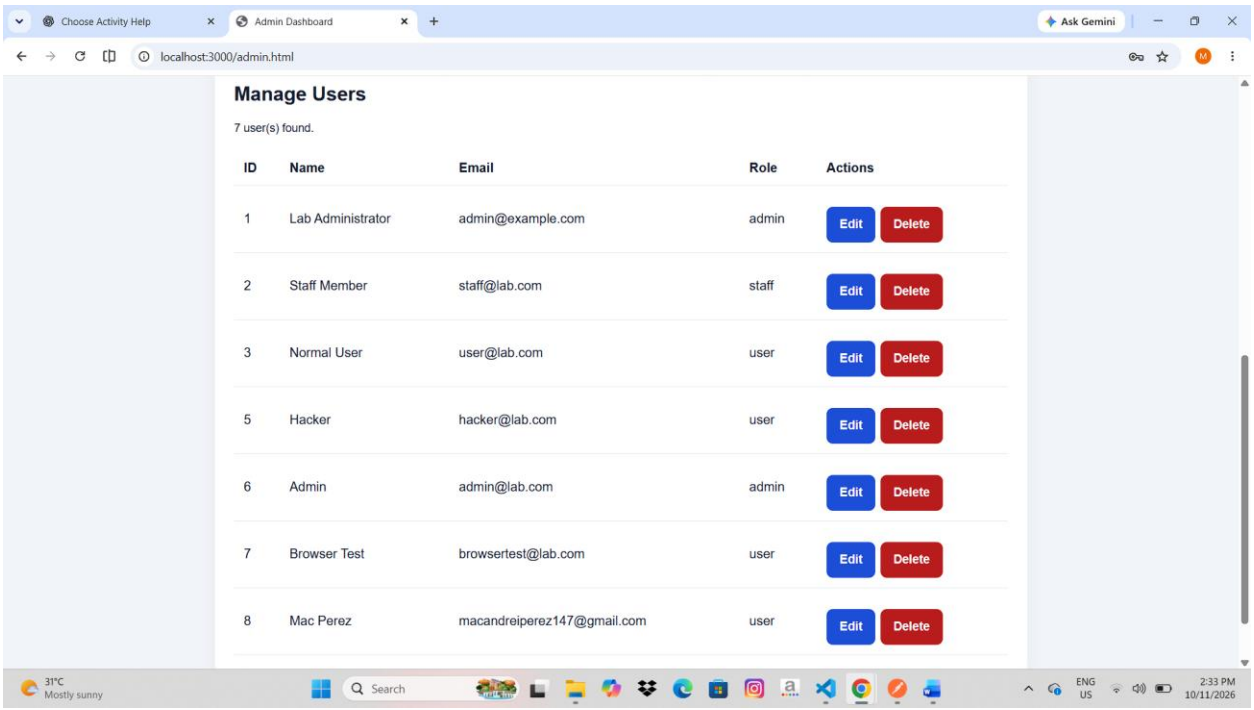
Task 7 — Browser Login & Role-Based Redirects (10 pts)

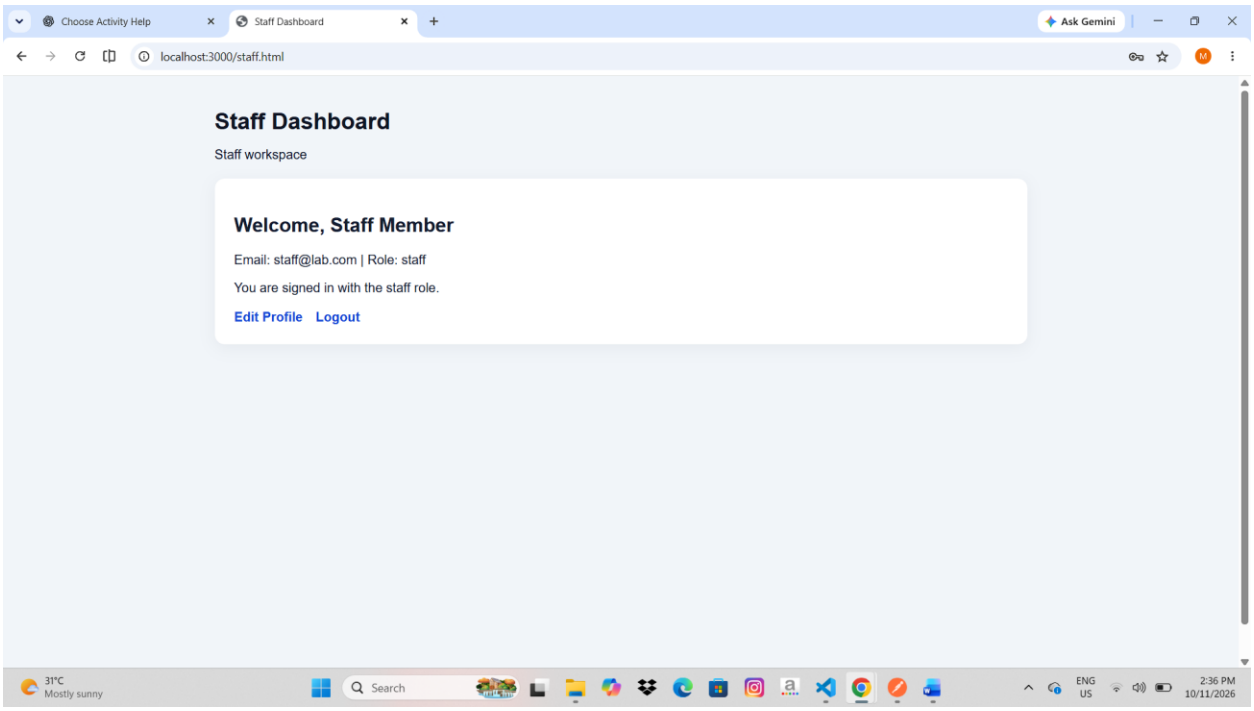
Test all three roles in the browser:

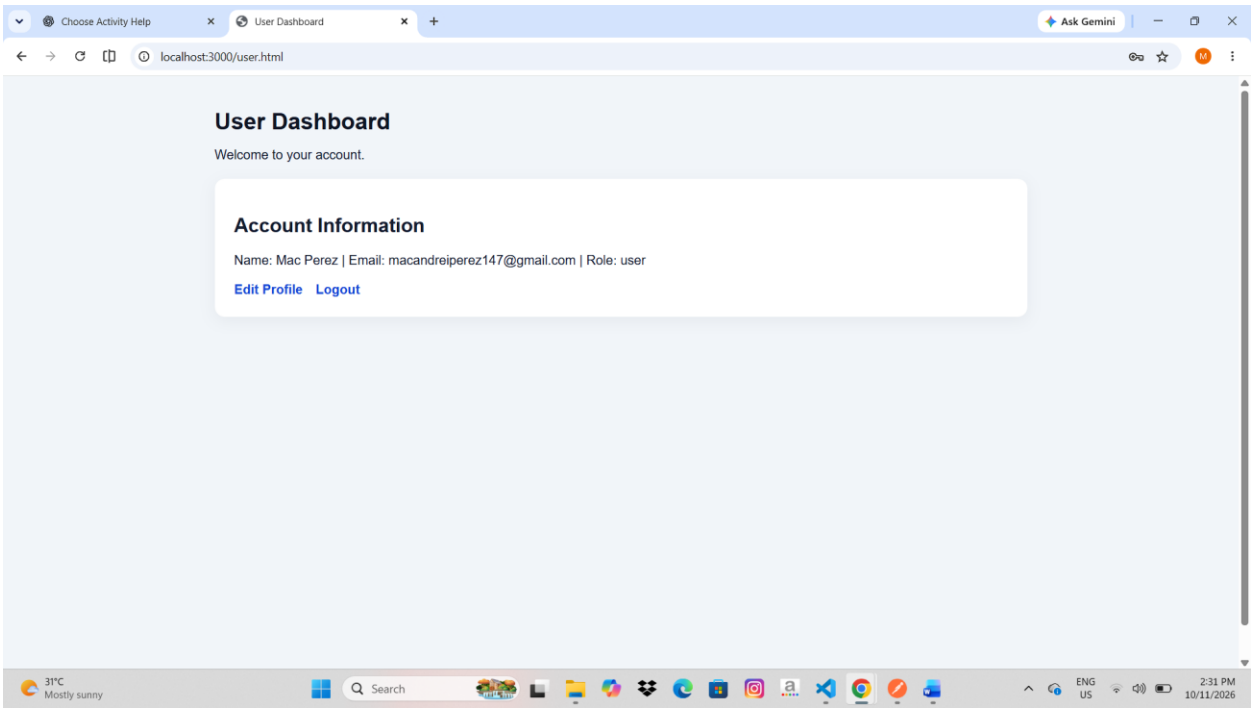
1. Login as **admin** → /admin.html with user table.
2. Logout.
3. Login as **staff** → /staff.html.
4. Logout.
5. Login as **user** → /user.html.
6. Try logging in with a wrong password → red error.

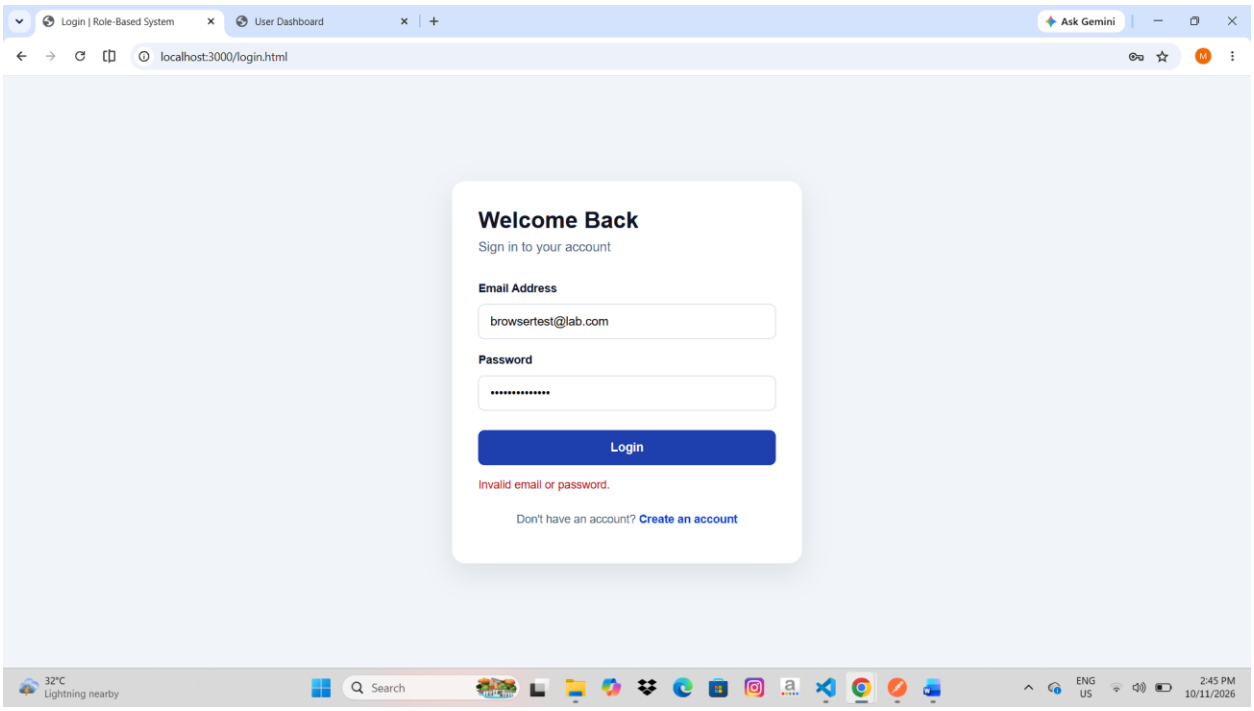
Deliverable: Screenshots of all three dashboards and the error.


Choose Activity Help
Admin Dashboard
Ask Gemini
localhost:3000/admin.html
Admin Dashboard
Welcome, Admin. Manage system users below.
Create User
Full Name
Email
Password (minimum 6 characters)
Role
User
Create User
31°C
Mostly sunny
Search
ENG
US
2:32 PM
10/11/2026


Choose Activity Help
Admin Dashboard
Ask Gemini
localhost:3000/admin.html
Manage Users
7 user(s) found.
ID
Name
Email
Role
Actions
1
Lab Administrator
admin@example.com
admin
Edit
Delete
2
Staff Member
staff@lab.com
staff
Edit
Delete
3
Normal User
user@lab.com
user
Edit
Delete
5
Hacker
hacker@lab.com
user
Edit
Delete
6
Admin
admin@lab.com
admin
Edit
Delete
7
Browser Test
browsertest@lab.com
user
Edit
Delete
8
Mac Perez
macandreiperez147@gmail.com
user
Edit
Delete
31°C
Mostly sunny
Search
ENG
US
2:33 PM
10/11/2026


Choose Activity Help
Staff Dashboard
Ask Gemini
localhost:3000/staff.html
Staff Dashboard
Staff workspace
Welcome, Staff Member
Email: staff@lab.com | Role: staff
You are signed in with the staff role.
Edit Profile
Logout
31°C
Mostly sunny
Search
ENG
US
2:36 PM
10/11/2026

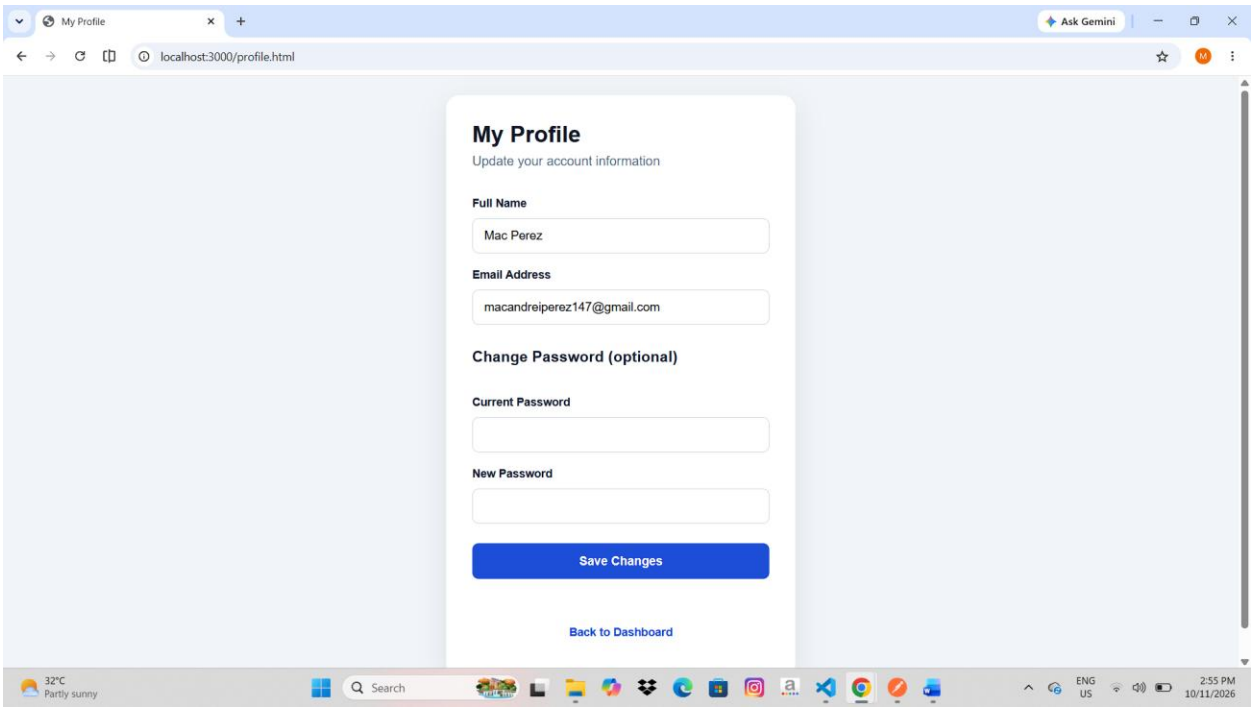

Choose Activity Help
User Dashboard
Ask Gemini
localhost:3000/user.html
User Dashboard
Welcome to your account.
Account Information
Name: Mac Perez | Email: macandreiperez147@gmail.com | Role: user
Edit Profile
Logout
31°C
Mostly sunny
Search
ENG
US
2:31 PM
10/11/2026

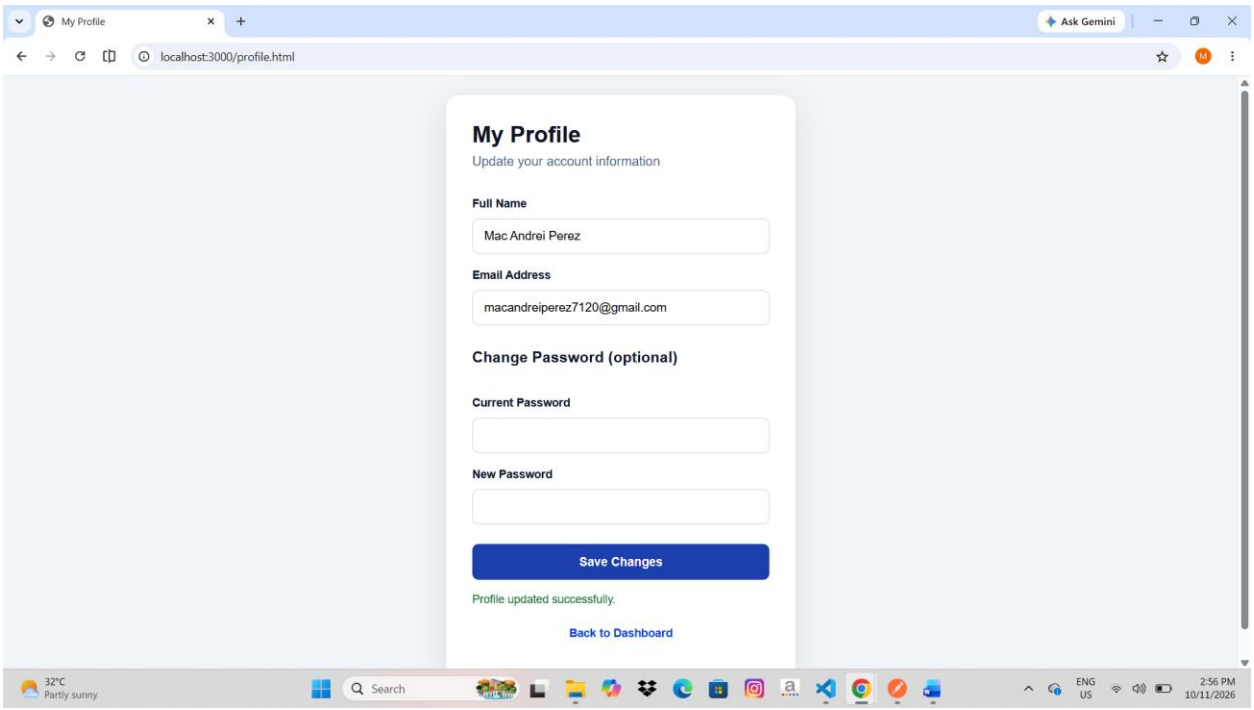


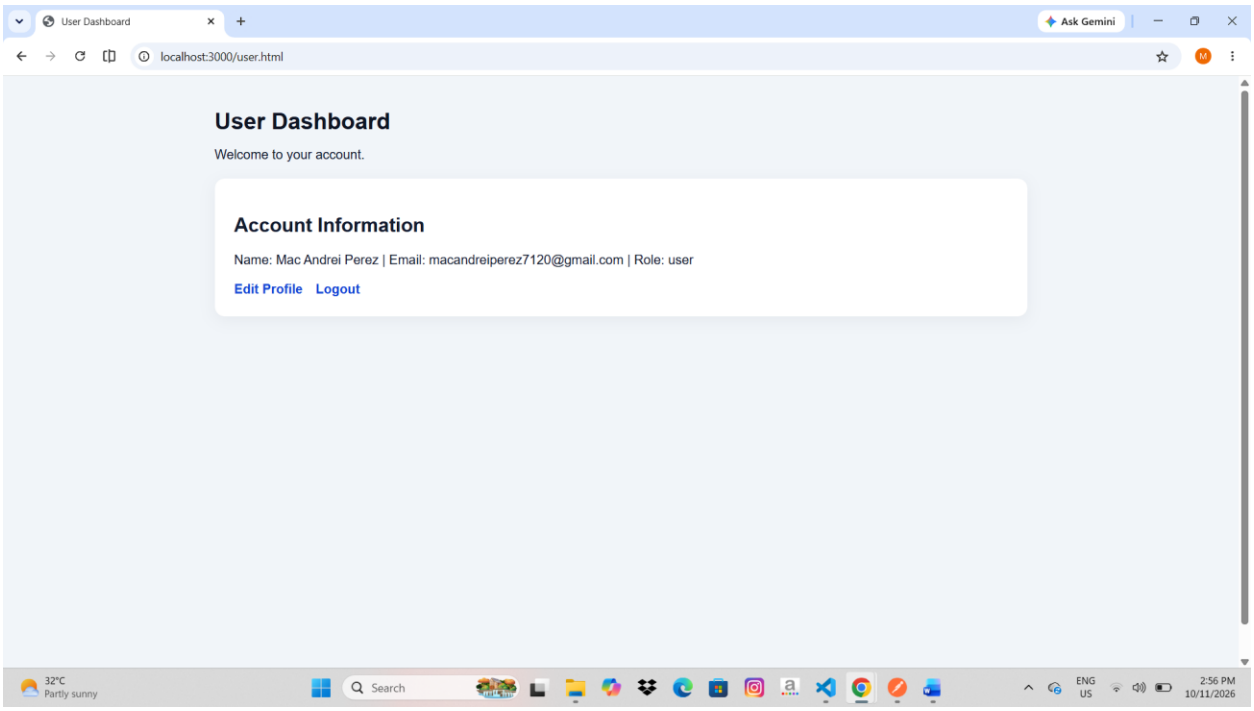
Task 8 — Profile Editing (10 pts)

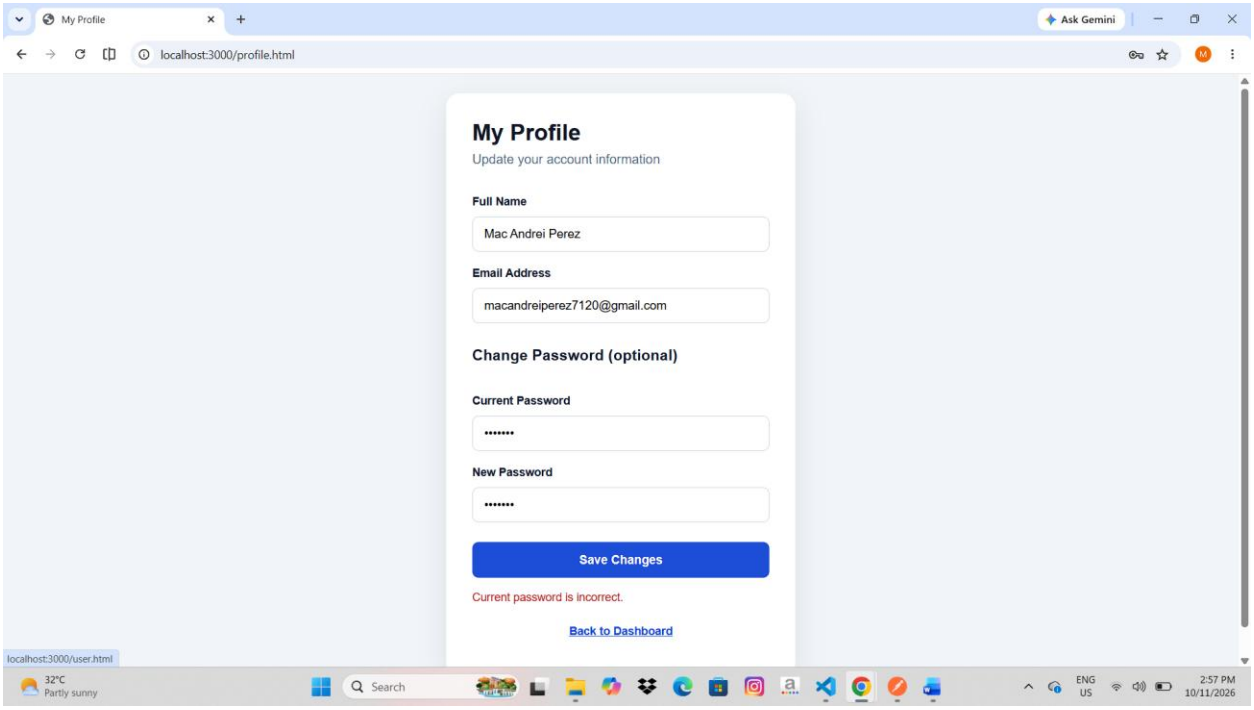
1. Log in as any user.
2. Go to **My Profile**.
3. Change name and email → save → confirm success.
4. Reload the dashboard → confirm updated name appears.
5. Change password with the **wrong current password** → error.
6. Change password with the **correct current password** → success.
7. Log out and log back in with the **new password** → works.

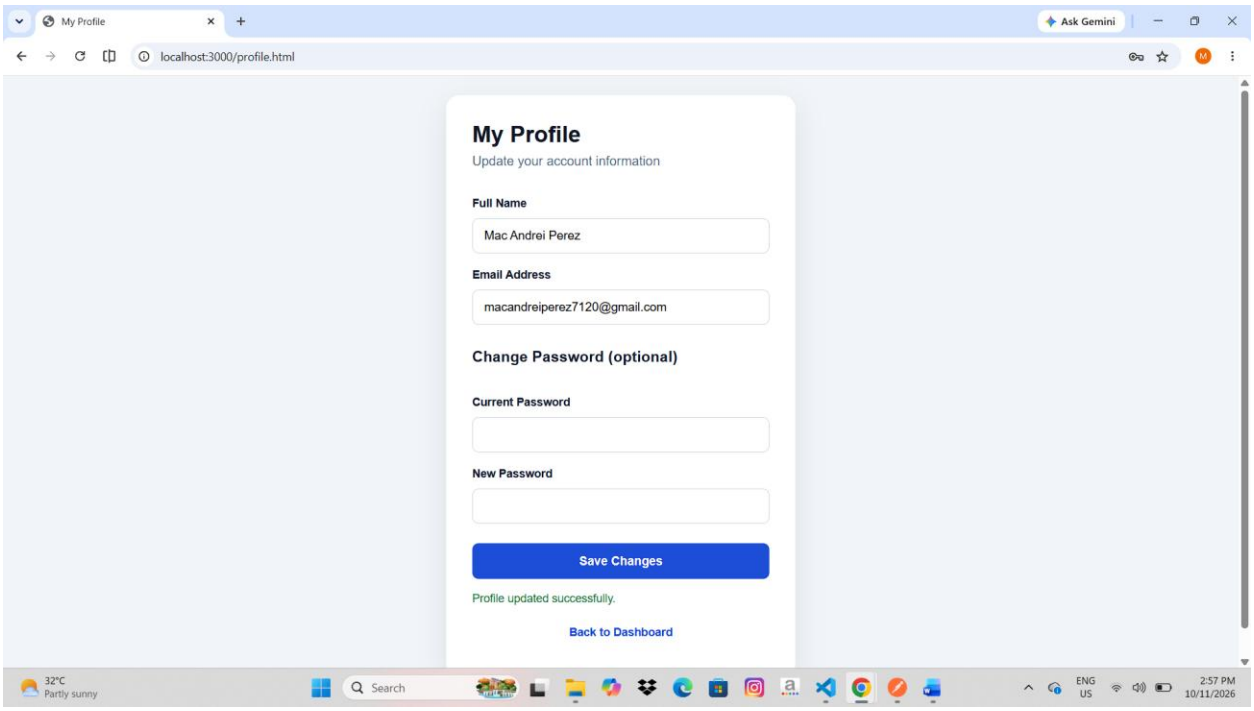
Deliverable: Screenshots of: profile page, successful update, wrong-password error, login with new password.

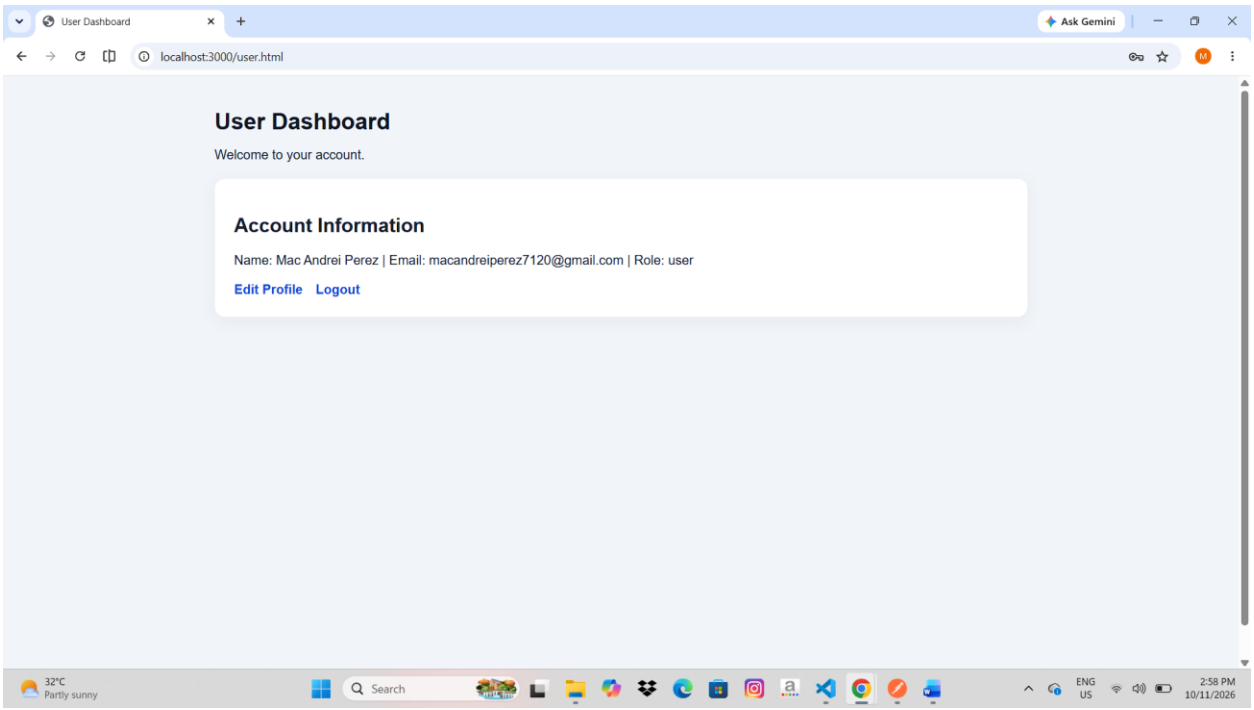

My Profile
Ask Gemini
localhost:3000/profile.html
My Profile
Update your account information
Full Name
Mac Perez
Email Address
macandreiperez147@gmail.com
Change Password (optional)
Current Password
New Password
Save Changes
Back to Dashboard
32°C
Partly sunny
Search
ENG
US
2:55 PM
10/11/2026


My Profile
Ask Gemini
localhost:3000/profile.html
My Profile
Update your account information
Full Name
Mac Andrei Perez
Email Address
macandreiperez7120@gmail.com
Change Password (optional)
Current Password
New Password
Save Changes
Profile updated successfully.
Back to Dashboard
32°C
Partly sunny
Search
ENG
US
2:56 PM
10/11/2026


User Dashboard
localhost:3000/user.html
Ask Gemini
User Dashboard
Welcome to your account.
Account Information
Name: Mac Andrei Perez | Email: macandreiperez7120@gmail.com | Role: user
Edit Profile
Logout
32°C
Partly sunny
Search
ENG
US
2:56 PM
10/11/2026


My Profile
localhost:3000/profile.html
Ask Gemini
My Profile
Update your account information
Full Name
Mac Andrei Perez
Email Address
macandreiperez7120@gmail.com
Change Password (optional)
Current Password
New Password
Save Changes
Current password is incorrect.
Back to Dashboard
localhost:3000/user.html
32°C
Partly sunny
Search
ENG
US
2:57 PM
10/11/2026

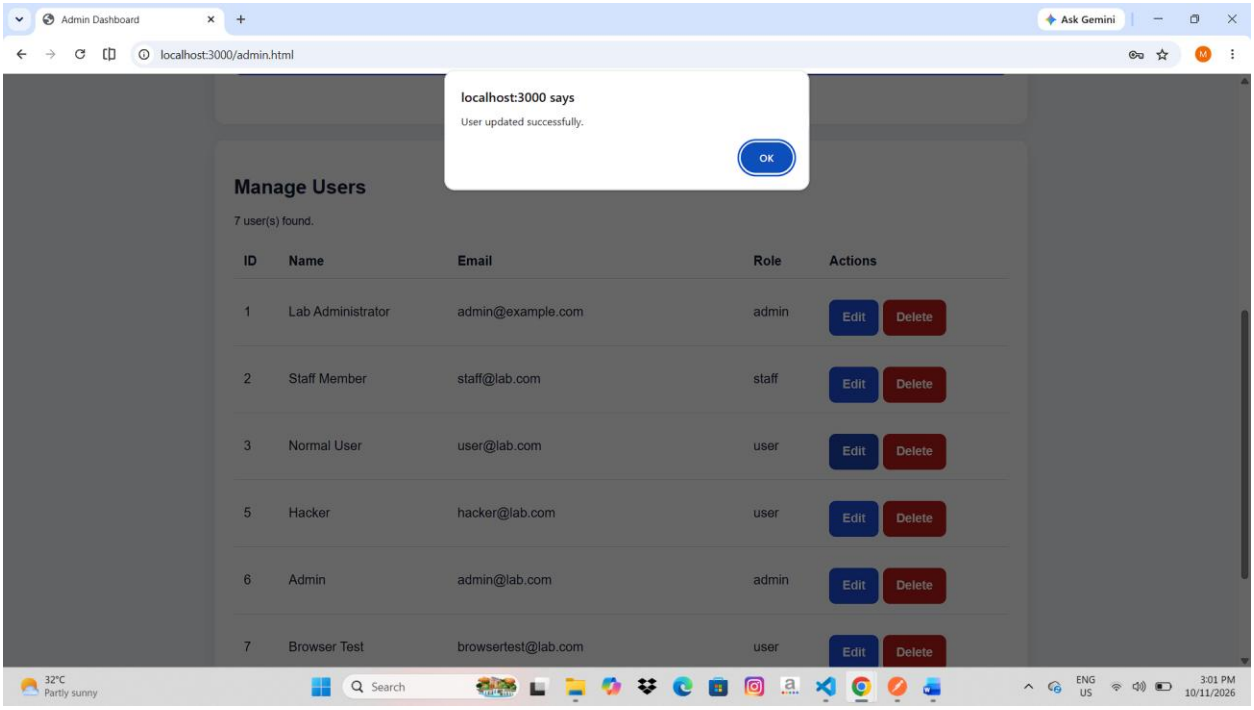


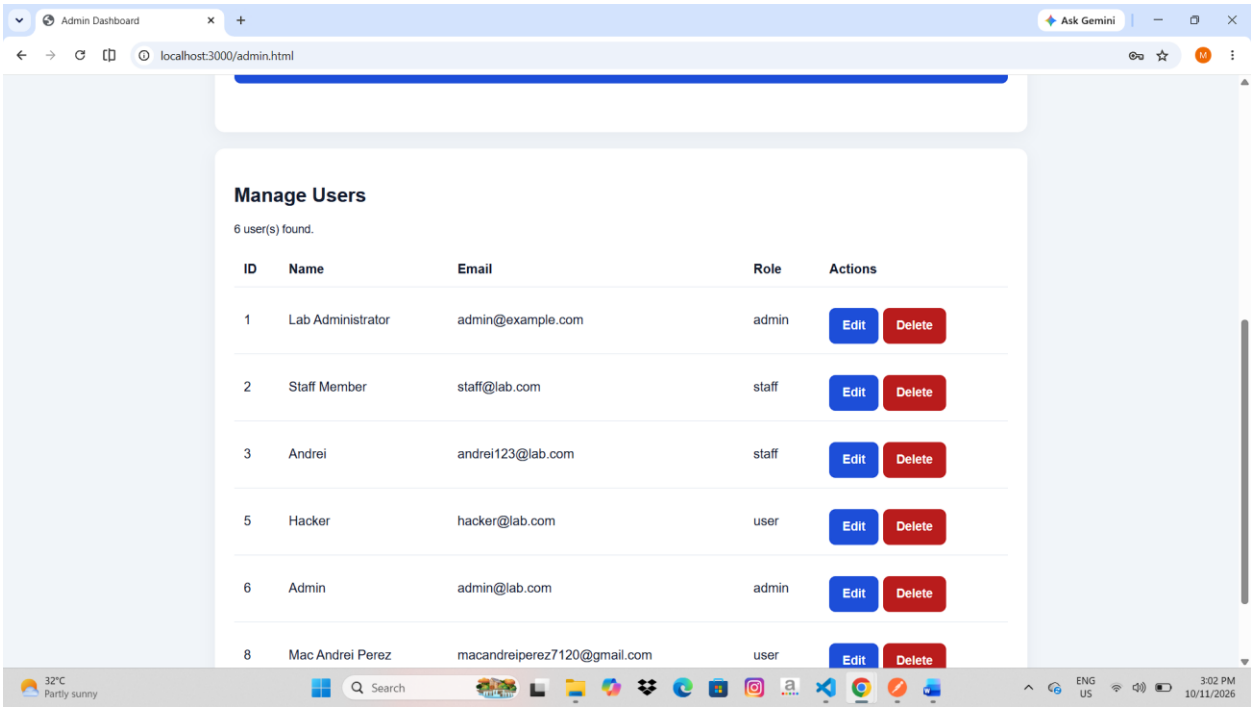


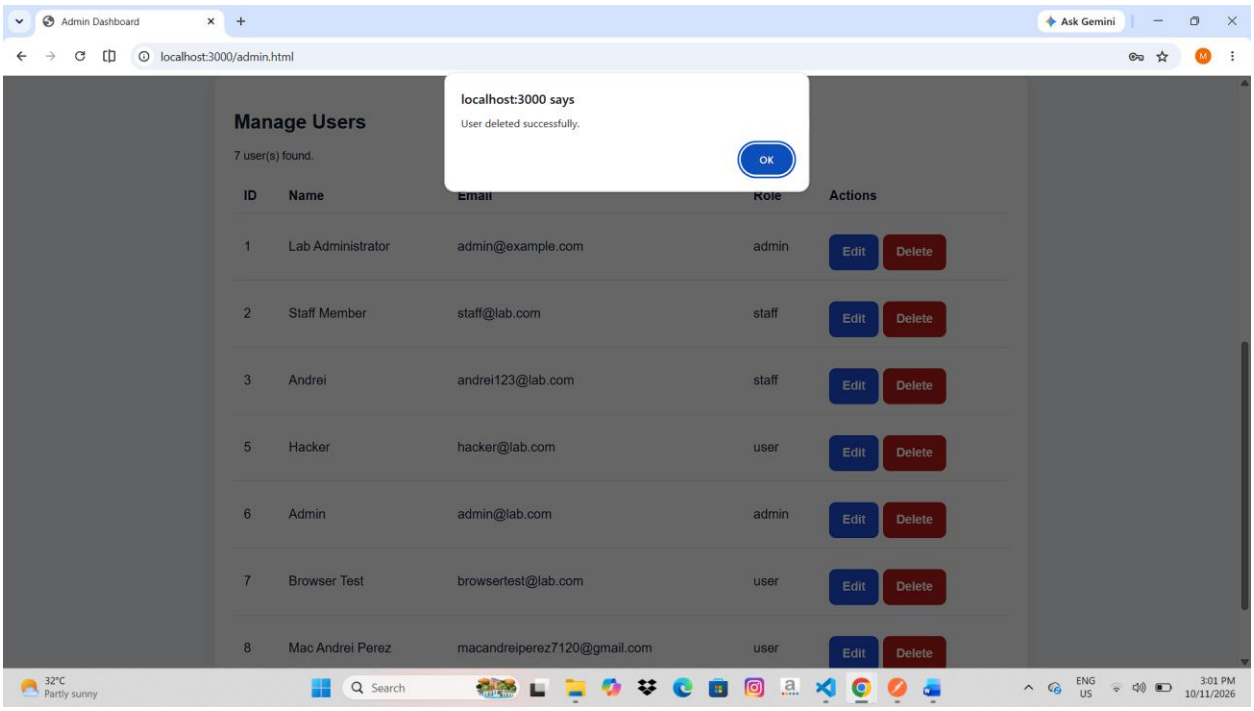
Task 9 — Admin User Management (10 pts)

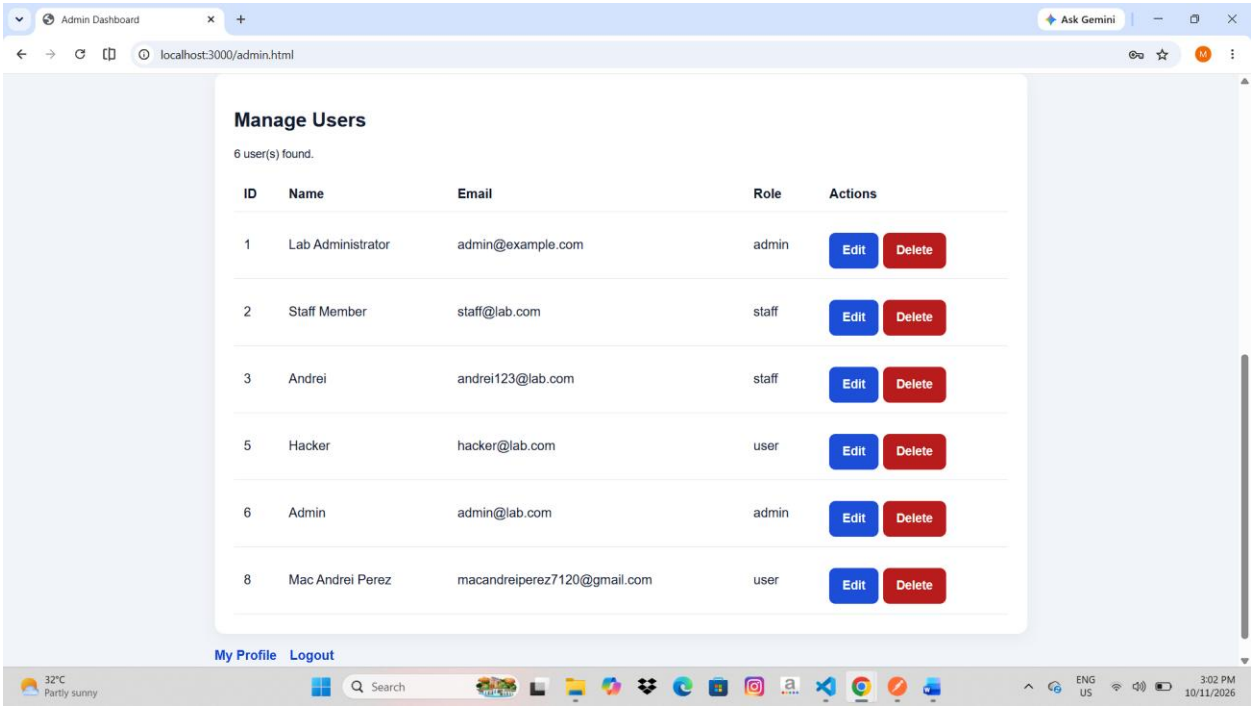
1. Log in as **admin**.
2. Click **Edit** on a user → update name, email, role (promote to staff).
3. Click **Delete** on a user → row disappears.
4. Try to **delete your own admin account** → must be blocked.

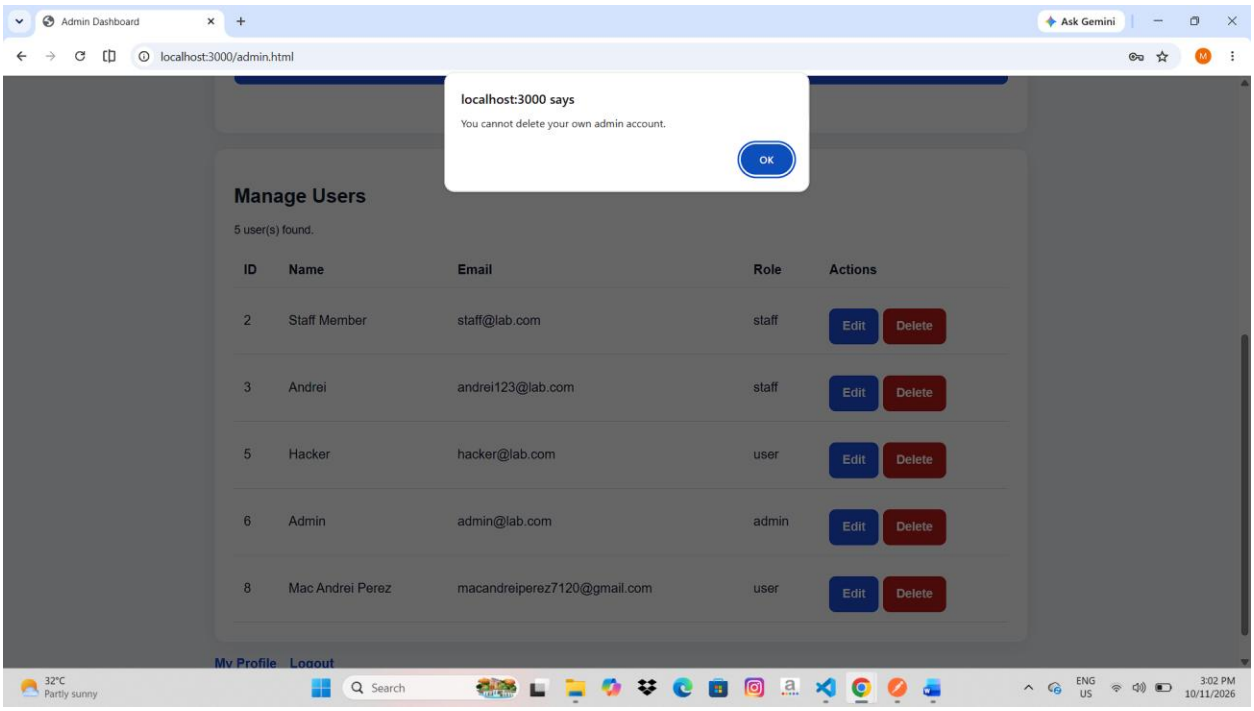
Deliverable: Screenshots of edit, delete, and blocked self-delete.






Admin Dashboard
Ask Gemini
localhost:3000/admin.html
localhost:3000 says
User deleted successfully.
OK
Manage Users
7 user(s) found.
ID Name Email Role Actions
1 Lab Administrator admin@example.com admin Edit Delete
2 Staff Member staff@lab.com staff Edit Delete
3 Andrei andrei123@lab.com staff Edit Delete
5 Hacker hacker@lab.com user Edit Delete
6 Admin admin@lab.com admin Edit Delete
7 Browser Test browsertest@lab.com user Edit Delete
8 Mac Andrei Perez macandreiperez7120@gmail.com user Edit Delete
32°C Partly sunny
Search
ENG US
3:01 PM 10/11/2026


Admin Dashboard
Ask Gemini
localhost:3000/admin.html
Manage Users
6 user(s) found.
ID Name Email Role Actions
1 Lab Administrator admin@example.com admin Edit Delete
2 Staff Member staff@lab.com staff Edit Delete
3 Andrei andrei123@lab.com staff Edit Delete
5 Hacker hacker@lab.com user Edit Delete
6 Admin admin@lab.com admin Edit Delete
8 Mac Andrei Perez macandreiperez7120@gmail.com user Edit Delete
My Profile Logout
32°C Partly sunny
Search
ENG US
3:02 PM 10/11/2026



Task 10 — Reflection Questions (10 pts)

Q1. In server.js, paste the exact block that decides redirectTo.

```
const redirectMap = {
  admin: '/admin.html',
  staff: '/staff.html',
  user: '/user.html'
};

return res.json({
  message: 'Login successful.',
  user: sanitizeUser(user),
  redirectTo: redirectMap[user.role]
});
```

Q2. What happens if a user manually sets sessionStorage.currentUser = {"role":"admin"} in DevTools? What does this reveal about client-side guards?

If the user manually changes sessionStorage.currentUser to contain the admin role, a dashboard that checks only this client-side value may allow the user to view the admin page. This reveals that client-side guards can be bypassed and are not enough to protect sensitive data or actions. The server must verify authentication and authorization for protected endpoints.

Q3. Why is the role forced to 'user' in /api/register?

The role is forced to 'user' to prevent people from registering themselves as administrators or staff by including a role field in their registration request. Privileged accounts should be created or assigned by an authorized administrator.

Q4. Why does /api/profile/:id require the **current password** before accepting a new one?

Requiring the current password helps verify that the person changing the password knows the account's existing credentials. It reduces the risk of someone who has access to an open session changing the account password without authorization.

Q5. Why does sanitizeUser strip passwords from every response?

The sanitizeUser function removes passwords and password hashes from API responses to protect sensitive information. Even if the server stores a hashed password, it should not send that hash to the browser or expose it through the API.

Q6. What happens to registered users when the server restarts? How is this fixed in real applications?

In this lab, users are stored in an in-memory array, so all registered users are lost when the Node.js server restarts. In real applications, user accounts are stored in a persistent database, such as MySQL or PostgreSQL, so they remain available after a restart.

Q7. How does this lab demonstrate **synchronous** client–server communication?

The lab demonstrates request-and-response communication: the client sends an HTTP request to the server and waits for a response before proceeding with the next step, such as displaying a result or redirecting to a dashboard. Technically, the HTTP requests are asynchronous in JavaScript, but the client handles the response before continuing the related operation.

Q8. What role does **JSON** play in every request and response?

JSON (JavaScript Object Notation) is the format used to exchange structured data between the client and server. The client sends information such as email, password, and registration details in JSON, and the server returns JSON containing results, user information, redirect paths, or error messages.

Q9. Give one example of **authentication** and one of **authorization** in this lab.

- Authentication: The /api/login route checks the user's email and password against the stored account credentials.
- Authorization: The application uses the user's role (admin, staff, or user) to determine which dashboard they should access. In a production application, the server must also enforce role permissions on protected API endpoints.

Q10. Propose **one** security improvement for production. Explain why.

I would add server-side authentication and role-based authorization middleware to protect the API endpoints. This ensures that only logged-in users with the correct permissions can access or modify protected data. It also prevents users from bypassing client-side checks by changing sessionStorage in the browser's DevTools.

Task 11 — Testing Checklist (5 pts)

Mark / or X when done:

#	Test Case	Expected	Done
1	Server starts with node server.js	Console shows URLs	/
2	Login page loads with styling	Renders correctly	/
3	Register page loads	Renders correctly	/
4	GET /api/users when empty	[]	/
5	POST /api/users creates admin	201, no password	/
6	POST /api/users creates staff	201, role=staff	/
7	POST /api/users creates user	201, role=user	/
8	GET /api/users	No passwords shown	/
9	POST /api/register valid	201, role=user	/
10	POST /api/register duplicate	409	/
11	POST /api/register short password	400	/
12	POST /api/register with role=admin	Role forced to user	/
13	POST /api/login admin	200, redirectTo=/admin.html	/

#	Test Case	Expected	Done
14	POST /api/login staff	200, redirectTo=/staff.html	/
15	POST /api/login user	200, redirectTo=/user.html	/
16	POST /api/login wrong password	401	/
17	POST /api/login unknown email	401 (same error)	/
18	POST /api/login missing password	400	/
19	Browser register → login → user dashboard	Works	/
20	Browser login as admin	Redirects to /admin.html	/
21	Browser login as staff	Redirects to /staff.html	/
22	Browser login as user	Redirects to /user.html	/
23	Logout clears sessionStorage	Returns to login	/
24	Profile page loads current data	Pre-filled	/
25	PUT /api/profile/:id name/email	200	/

#	Test Case	Expected	Done
26	PUT profile with wrong current password	401	/
27	PUT profile with correct current password	200	/
28	Login with new password	200	/
29	Admin edit user	Table updates	/
30	Admin delete user	Row removed	/
31	Admin delete own account	Blocked	/
32	GET deleted user	404	/
33	Restart server — all users cleared	Login fails	/

VII. Deliverables

A. Source Code Files

text

```
role-login-lab/
├── server.js
├── package.json
└── public/
    ├── login.html
    ├── register.html
    ├── profile.html
    ├── admin.html
    ├── staff.html
    ├── user.html
    └── style.css
```

Do not include node_modules/.

B. Must Contain

- **Pre-Lab Answers** (Q1–Q5)
- **Task 10 Reflection Answers** (Q1–Q10)
- **Task 11 Testing Checklist** (33 items marked)
- Any notes, bugs you fixed, or observations

C. Screenshots Folder

Create screenshots/ and include:

Task	Screenshot
1	Terminal (server running) + login page
2	3 POST /api/users + GET users
3	5 POST /api/register scenarios
4	6 POST /api/login scenarios
5	GET one, PUT update, DELETE, GET 404
6	Registration page, success, duplicate error
7	Admin, staff, user dashboards + wrong password error

Task	Screenshot
8	Profile page, updated profile, wrong-password error, login with new password
9	Admin edit prompts, delete, blocked self-delete

D. Submission Format

Choose **one:**

- **ZIP file** named role-login-lab.zip
- **Public GitHub repository link**

VIII. Grading Rubric

Section	Description	Points
Pre-Lab	5 conceptual questions	10
Task 1	Server runs, pages load	5
Task 2	Users created via Postman	10
Task 3	Registration endpoint tested (5 cases)	10
Task 4	Login endpoint tested (6 cases)	10
Task 5	Full CRUD tested in Postman	10
Task 6	Browser registration flow	10
Task 7	Role-based redirects verified	10
Task 8	Profile editing tested	10
Task 9	Admin user management tested	10
Task 10	10 reflection questions	10
Task 11	Testing checklist (33 items)	5
Subtotal		**110**
Adjusted		**100**

IX. Bonus Task (+10 pts) — Hash Passwords with bcrypt

Goal: Never store plain-text passwords.

1. Install bcrypt:

bash

```
npm install bcrypt
```

2. Require it in server.js:

Javascript

```
const bcrypt = require('bcrypt');
```

3. In POST /api/users, POST /api/register, and PUT /api/profile/:id, hash before storing:

javascript

```
const hashed = await bcrypt.hash(password, 10);
newUser.password = hashed;
```

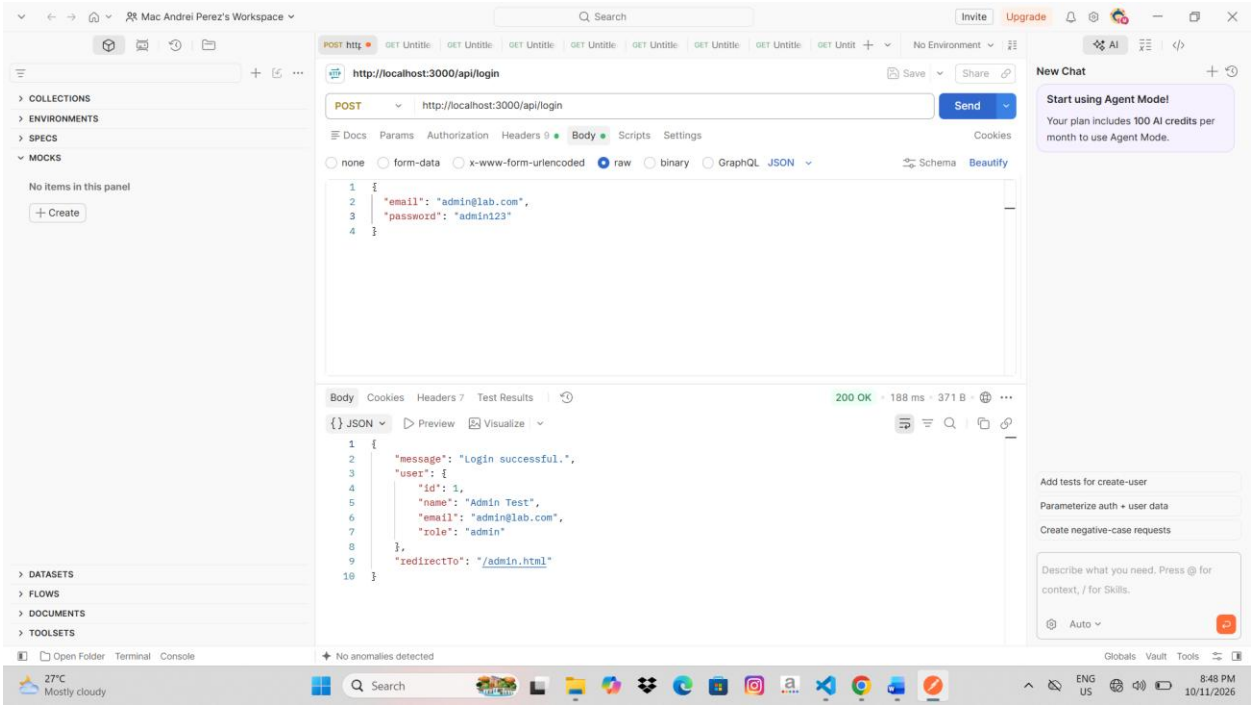
4. In POST /api/login, compare with bcrypt:

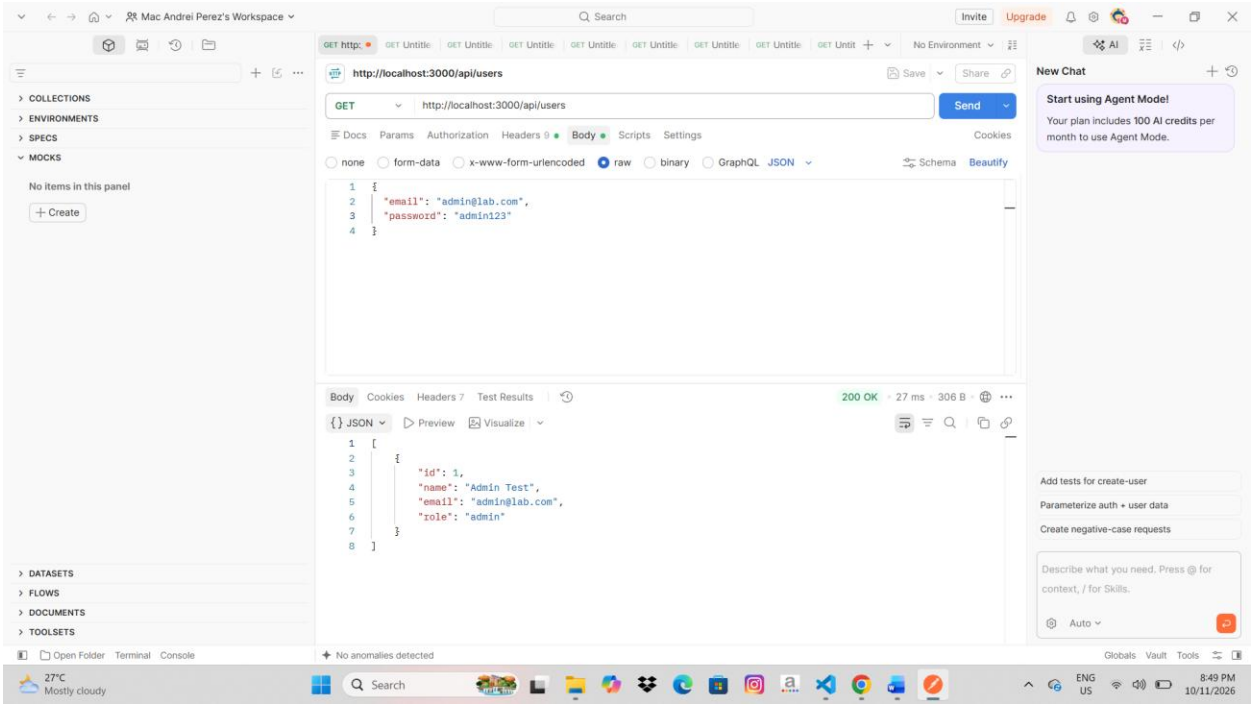
javascript

```
const match = await bcrypt.compare(password, user.password);
if (!match) return res.status(401).json({ error: 'Invalid email or password.' });
```

5. In PUT /api/profile/:id, verify current password with bcrypt.compare and hash the new one.
6. Recreate all users via Postman — old plain-text records will no longer match.

Deliverable: Updated server.js + screenshots showing login still works and GET /api/users never reveals hashes.




Mac Andrei Perez's Workspace
Search
Invite
Upgrade
COLLECTIONS
ENVIRONMENTS
SPECS
MOCKS
No items in this panel
Create
DATASETS
FLOWS
DOCUMENTS
TOOLSETS
http://localhost:3000/api/users
GET
http://localhost:3000/api/users
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"email": "admin@lab.com",
"password": "admin123"
}
Body
Cookies
Headers 7
Test Results
200 OK
27 ms
306 B
JSON
Preview
Visualize
[
{
"id": 1,
"name": "Admin Test",
"email": "admin@lab.com",
"role": "admin"
}
]
New Chat
Start using Agent Mode!
Your plan includes 100 AI credits per month to use Agent Mode.
Add tests for create-user
Parameterize auth + user data
Create negative-case requests
Describe what you need. Press @ for context, / for Skills.
Auto
Open Folder
Terminal
Console
No anomalies detected
Globals
Vault
Tools
27°C
Mostly cloudy
Search
ENG
US
8:49 PM
10/11/2026

X. Summary

By completing this activity, you built a **complete full-stack integration demo** using only Node.js and the browser:

- Full REST API CRUD for users
- Public self-registration with forced user role
- Role-based login with server-controlled redirects
- Self-service profile editing with password verification
- Admin user management with edit/delete from the UI
- Postman testing for every endpoint
- Browser integration via fetch() and sessionStorage

Key insight: The **server** — not the client — is the source of truth for authentication and role-based decisions. Client-side checks (sessionStorage) are UX conveniences only; they are trivially bypassed. Real systems must add **token-based authentication (JWT)**, **hashed passwords (bcrypt)**, and **persistent storage (database)** — the natural next step in your learning path.

End of Final Laboratory Activity